CONFIDENTIAL

SDK USER MANUAL
（FOR INTERNAL USE ONLY）

CONFIDENTIAL

2026-9

Revision History

No.	Date	Note	version
01	2026.9	Initial release.	V1.0.0
02	2026.10	1.Added section 7.22 Printer APIs; 2.Added section 7.1.41 Lib_realloc memory reallocation API; 3.Modified section 7.8.15 Lib_WlsPing parameters.	V1.1.0

CONFIDENTIAL

Contents

1 Introduction 1
2 Framework 2
3 Resource Management 3
3.1 Flash 3
3.2 RAM 3
4 Function Naming and Data Type 4
4.1 Function naming rules 4
5 Key System 5
5.1 MK/SK Key System 5
5.2 DUPKT Key System 6
6 TMS Upgrade 8
7 API Description 9
7.1 System Interface 9
7.1.1 Lib_OsApiInit 9
7.1.2 Lib_SysInit 9
7.1.3 Lib_AppInit 9
7.1.4 Lib_DelayMs 10
7.1.5 Lib_RtcToUtcSec 10
7.1.6 Lib_UtcSecToRtc 10
7.1.7 Lib_SetDateTime 11
7.1.8 Lib_GetDateTime 11
7.1.9 Lib_SyncDateTime 11
7.1.10 Lib_TimeGetUnixTime 12
7.1.11 Lib_SysGetTimer 12
7.1.12 Lib_ReadSn 13
7.1.13 Lib_GetPN 13

CONFIDENTIAL

7.1.14 Lib_GetDevState 13
7.1.15 Lib_Reboot 13
7.1.16 Lib_ShutDown 14
7.1.17 Lib_SysSleep 14
7.1.18 Lib_FactoryRecovery 14
7.1.19 Lib_ExtPower 15
7.1.20 Lib_PowerChargingCtrl 15
7.1.21 Lib_GetPwrKeyStatus 15
7.1.22 Lib_GetPowerOnCause 15
7.1.23 Lib_MaintenanceMode 16
7.1.24 Lib_WDGReloadCounter 16
7.1.25 Lib_TamperIsOccured 16
7.1.26 Lib_KbGetKey 17
7.1.27 Lib_Beep 17
7.1.28 Lib_SdkCallbackInit 17
7.1.29 Lib_PrintfHeapInfo 17
7.1.30 Lib_malloc 18
7.1.31 Lib_free 18
7.1.32 Lib_GetHardState 18
7.1.33 Lib_CheckUpgradeFlag 19
7.1.34 Lib_SetInfo 19
7.1.35 Lib_GetInfo 20
7.1.36 Lib_XmlCvtIdToStr 20
7.1.37 Lib_Trace 21
7.1.38 Lib_TraceBuf 21
7.1.39 Lib_TraceLogConfig 21
7.1.40 Lib_GetVersion 22
7.1.41 Lib_realloc 22
7.2 Algorithm 23
7.2.1 Lib_AlgDes 23

CONFIDENTIAL

7.2.2 Lib_AlgAes 23
7.2.3 Lib_AlgHash(SHA-1) 24
7.2.4 Lib_AlgHash256 24
7.2.5 Lib_AlgAes256gcmEncrypt 25
7.2.6 Lib_AlgAes256gcmDecrypt 25
7.2.7 Lib_AlgMD5 26
7.2.8 Lib_AlgGetRandom 26
7.2.9 Lib_AlgRsa 27
7.2.10 Lib_AlgRsaKeyPairGen 27
7.2.11 Lib_AlgRsaKeyPairVerify 27
7.2.12 Lib_AlgBase64Encode 28
7.2.13 Lib_AlgBase64Decode 28
7.2.14 Lib_AlgGetCrc16 28
7.3 LCD Drive Interface 29
7.3.1 Lib_LcdSetBrightLevel 29
7.3.2 Lib_LcdClrScreen 29
7.3.3 Lib_LcdRefresh 29
7.3.4 Lib_LcdDispMemcpy 29
7.3.5 Lib_LcdParaSet 30
7.3.6 Lib_LcdParaGet 30
7.4 LED 31
7.4.1 Lib_LedControl 31
7.5 Card Drive Interface 31
7.5.1 Lib_MscOpen 31
7.5.2 Lib_MscClose 31
7.5.3 Lib_MscRead 32
7.5.4 Lib_IccOpen 32
7.5.5 Lib_IccClose 33
7.5.6 Lib_IccInquiry 33
7.5.7 Lib_IccCommand 33

CONFIDENTIAL

7.5.8 Lib_IccIsPresent 34
7.5.9 Lib_PiccOpen 34
7.5.10 Lib_PiccClose 34
7.5.11 Lib_PiccInquiry 35
7.5.12 Lib_PiccCommand 35
7.5.13 Lib_PiccRemove 36
7.6 Touch Screen Interface 36
7.6.1 Lib_TpGetCoordinates 36
7.6.2 Lib_TpGetESignData 36
7.6.3 Lib_TpCalibrate 37
7.6.4 Lib_TpGetKey 37
7.7 Code Scan Module 38
7.7.1 Lib_CamInit 38
7.7.2 Lib_CamOpen 38
7.7.3 Lib_CamClose 38
7.7.4 Lib_CamGetResult 38
7.7.5 Lib_CamDisp 39
7.7.6 Lib_CamLed 39
7.8 GPRS Communication Interface 39
7.8.1 Lib_WlsInit 39
7.8.2 Lib_WlsGetInitStatus 40
7.8.3 Lib_WlsCheckSignal 40
7.8.4 Lib_WlsCheckSim 40
7.8.5 Lib_WlsGetInfo 41
7.8.6 Lib_WlsGetBSinfo 41
7.8.7 Lib_WlsSetDns 41
7.8.8 Lib_WlsGetDns 42
7.8.9 Lib_WlsDnsResolve 42
7.8.10 Lib_WlsGetApn 42
7.8.11 Lib_WlsSetApn 43

CONFIDENTIAL

7.8.12 Lib_WlsSetAdaptiveAPN 43
7.8.13 Lib_WlsGetAPNList 44
7.8.14 Lib_WlsGetApnAuth 44
7.8.15 Lib_WlsPing 44
7.8.16 Lib_WlsGetNetMode 45
7.8.17 Lib_WlsSetNetMode 45
7.8.18 Lib_WlsSetAirplaneMode 45
7.8.19 Lib_WlsGetOperatorList 46
7.8.20 Lib_WlsSetOperator 46
7.8.21 Lib_WlsGetOperator 46
7.8.22 Lib_WlsNetActRefresh 47
7.9 WiFi Communication Module 47
7.9.1 Lib_WifiNetworkUp 47
7.9.2 Lib_WifiNetworkDown 47
7.9.3 Lib_WifiNetworkScan 48
7.9.4 Lib_WifiGetAP 48
7.9.5 Lib_WifiConnectAp 48
7.9.6 Lib_WifiDisAp 49
7.9.7 Lib_WifiGetApStatus 49
7.9.8 Lib_WifiCheckSignal 50
7.9.9 Lib_WifiSetAutoConnectFlag 50
7.9.10 Lib_WifiGetLinkAp 50
7.9.11 Lib_WifiGetInfo 51
7.9.12 Lib_WifiSetInfo 51
7.9.13 Lib_WifiGetVersion 51
7.10 Battery Manage Interfaces 52
7.10.1 Lib_BatRefreshDisCapacity 52
7.10.2 Lib_BatGetCapacity 52
7.10.3 Lib_BatDischargeLowCap 52
7.10.4 Lib_BatGetPowerOffVol 53

CONFIDENTIAL

7.10.5 Lib_BatGetVoltage 53
7.10.6 Lib_BatGetChargeState 53
7.10.7 Lib_BatGetLibVersion 54
7.11 Network Communication Interfaces 54
7.11.1 Lib_NetSocket 54
7.11.2 Lib_NetConnect 54
7.11.3 Lib_NetSend 55
7.11.4 Lib_NetRecv 55
7.11.5 Lib_NetClose 55
7.11.6 Lib_NetGetStatus 56
7.11.7 Lib_NetSetCfg 56
7.11.8 Lib_NetDns 56
7.11.9 Lib_NetOpenEx 57
7.11.10 Lib_NetConnectEx 57
7.11.11 Lib_NetStatusEx 58
7.11.12 Lib_NetReadEx 58
7.11.13 Lib_NetWriteEx 59
7.11.14 Lib_NetCloseEx 59
7.11.15 Lib_NetResetEx 59
7.12 Pinpad Interface 60
7.12.1 Lib_PciGetMac 60
7.12.2 Lib_PciAlgDes 61
7.12.3 Lib_PciAlgAes 62
7.12.4 Lib_PciWriteKey 62
7.12.5 Lib_PciGetPin 63
7.13 Files Operation 63
7.13.1 Lib_FileOpen 63
7.13.2 Lib_FileRead 64
7.13.3 Lib_FileWrite 64
7.13.4 Lib_FileClose 64

CONFIDENTIAL

7.13.5 Lib_FileSeek 65
7.13.6 Lib_FileSize 65
7.13.7 Lib_FileRemove 65
7.13.8 Lib_FileFreeSize 66
7.13.9 Lib_FileMkdir 66
7.13.10 Lib_FileOpendir 66
7.13.11 Lib_FileReaddir 66
7.13.12 Lib_FileClosedir 67
7.13.13 Lib_FileGetAllFiles 67
7.13.14 Lib_FileClearPathAllFiles 67
7.14 TMS Upgrade 67
7.14.1 Lib_TmsDeleteSoft 67
7.14.2 Lib_TmsDownloadRemote 68
7.14.3 Lib_TmsGetLibVersion 68
7.15 EMV Interface 69
7.15.1 Lib_EmvKernelInit 69
7.15.2 Lib_EmvProcess 69
7.15.3 Lib_EmvAfterOnlineProcess 69
7.15.4 Lib_EmvGetKernelProcessResult 70
7.15.5 Lib_EmvGetTlv 70
7.15.6 Lib_EmvSetTlv 71
7.15.7 Lib_EmvGetEcBalance 71
7.15.8 Lib_EmvGetLibVersion 71
7.15.9 Lib_EarseTerminalAIDandCapk 72
7.15.10 Lib_GetAIDandCapkTotal 72
7.15.11 Lib_GetTerminalAID 72
7.15.12 Lib_GetTerminalCapk 73
7.15.13 Lib_SetTerminalAIDAndCapk 73
7.16 OS Interface (OSI) APIs 74
7.16.1 Lib_osiThreadCreate 74

CONFIDENTIAL

7.16.2 Lib_osiThreadDelete 74
7.16.3 Lib_osiThreadCurrent 75
7.16.4 Lib_osiMutexCreate 75
7.16.5 Lib_osiMutexDelete 75
7.16.6 Lib_osiMutexLock 75
7.16.7 Lib_osiMutexTryLock 76
7.16.8 Lib_osiMutexUnlock 76
7.16.9 Lib_osiSemaphoreCreate 76
7.16.10 Lib_osiSemaphoreDelete 77
7.16.11 Lib_osiSemaphoreRelease 77
7.16.12 Lib_osiSemaphoreAcquire 77
7.16.13 Lib_osiSemaphoreTryAcquire 78
7.16.14 Lib_osiCreatQueue 78
7.16.15 Lib_osiQueueSend 78
7.16.16 Lib_osiQueueReceive 79
7.16.17 Lib_osiTimerCreate 79
7.16.18 Lib_osiTimerDelete 80
7.16.19 Lib_osiTimerStart 80
7.16.20 Lib_osiTimerStop 80
7.17 Data Conversion 81
7.17.1 Lib_Asc2Bcd 81
7.17.2 Lib_Bcd2Asc 81
7.17.3 Lib_Bcd2Int 81
7.17.4 Lib_Int2Bcd 82
7.17.5 Lib_HextoStr 82
7.17.6 Lib_AscNum2bcd 83
7.18 LVGL Custom Interfaces 83
7.18.1 Lib_LvglInit 83
7.18.2 Lib_LvglGetFont 83
7.18.3 Lib_LvglTask 84

CONFIDENTIAL

7.18.4 Lib_LvglLock 84
7.18.5 Lib_LvglSleep 84
7.18.6 Lib_LvglWakeUp 84
7.19 LVGL Standard Interface 85
7.20 cJSON Standard Interface 85
7.21 mxml Standard Interface 85
7.22 Print Interface 85
7.22.1 Lib_PrnParaDefaultInit 85
7.22.2 Lib_PrnOpen 86
7.22.3 Lib_PrnClose 86
7.22.4 Lib_PrnCheckStatus 86
7.22.5 Lib_PrnSetSpace 87
7.22.6 Lib_PrnSetFont 87
7.22.7 Lib_PrnGetFont 87
7.22.8 Lib_PrnSetGrade 88
7.22.9 Lib_PrnGetGrade 88
7.22.10 Lib_PrnSetForwardDistance 88
7.22.11 Lib_PrnPaperForward 88
7.22.12 Lib_PrnGetStrWidth 89
7.22.13 Lib_PrnStr 89
7.22.14 Lib_PrnStrEx 90
7.22.15 Lib_PrnEsig 90
7.22.16 Lib_PrnBmp 91
7.22.17 Lib_PrnXCode 91
8 Appendix 93

CONFIDENTIAL

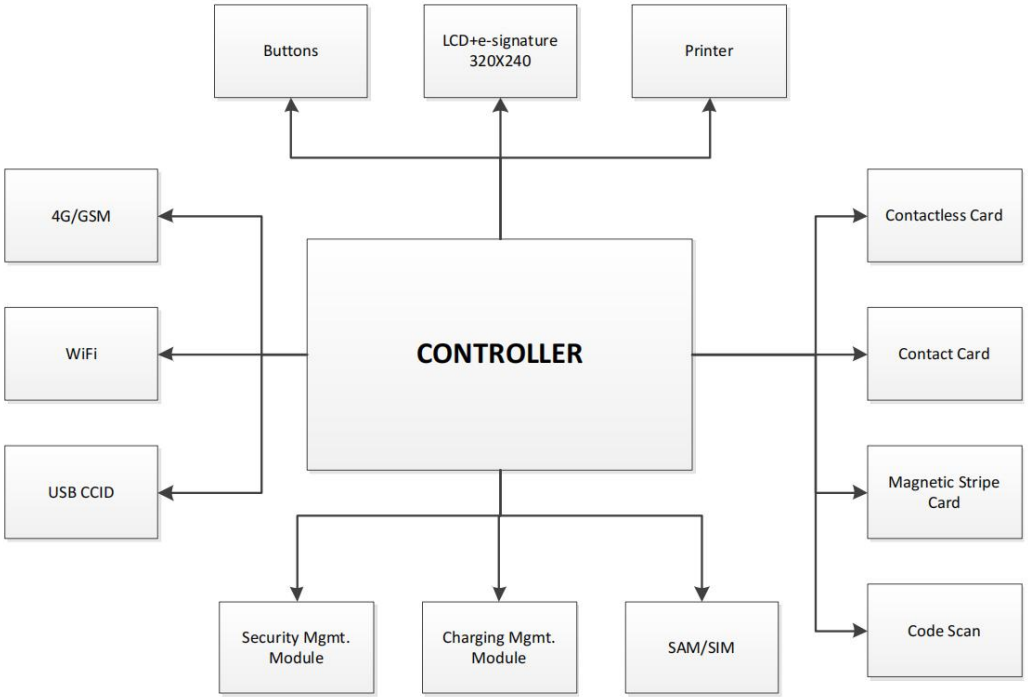
1 Introduction

This document is used to guide how to develop POS applications. The document describes in detail software resources, application modules and related interface function descriptions.

The main hardware modules involved include: buttons, LCD (TFT320x240), touch screen, buzzer, SAM card, contact IC card, NFC card, magnetic card, printer, scan code, charging management, etc.; support multiple communications Mode (such as CCID, GPRS, WiFi).

CONFIDENTIAL

2 Framework

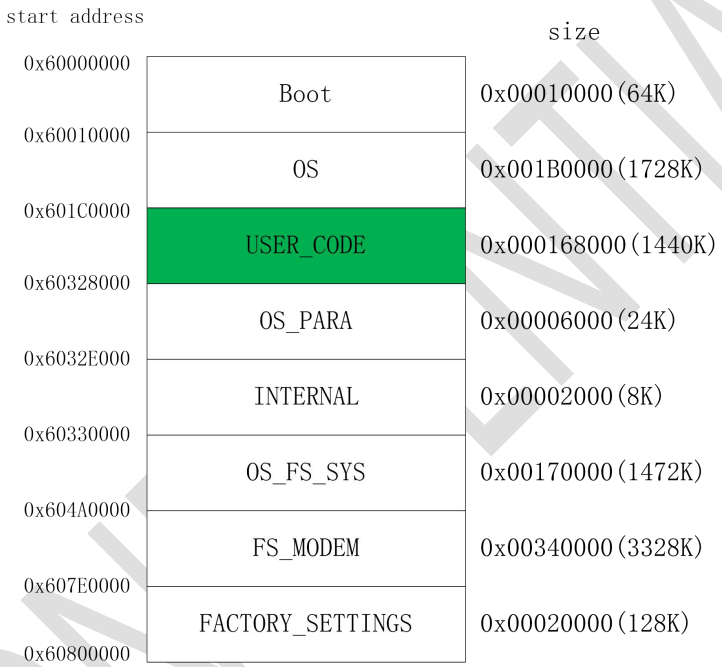


CONFI

3 Resource Management

3.1 Flash

The Flash memory consists of two parts: internal Flash (8MB) and external SPI Flash (8MB/16MB).
The internal Flash space is shown in the figure. User code is provided to users, with a size of 1440k. Other areas are used by POS manufacturers.



The external SPI Flash can be used to store picture resources, language xml, download files, or trade data etc.

3.2 RAM

After device bootup, a maximum of 3MB uninitialized heap space for users and 640KB space for global variables.

	Size	Description
HEAP	0x300000 (3MB)	Heap space, Lib_malloc for memory allocation
RAM	0xA0000 (640KB)	Global variable storage

CONFIDENTIAL

4 Function Naming and Data Type

4.1 Function naming rules

- **Unified format of function interface：**

  Lib_MMMMFFFF

  Lib: device API interface

  MMMM： module full name or abbreviation；

  FFFF： function and action description of function interface；

- **Naming of the module full name or abbreviation：**

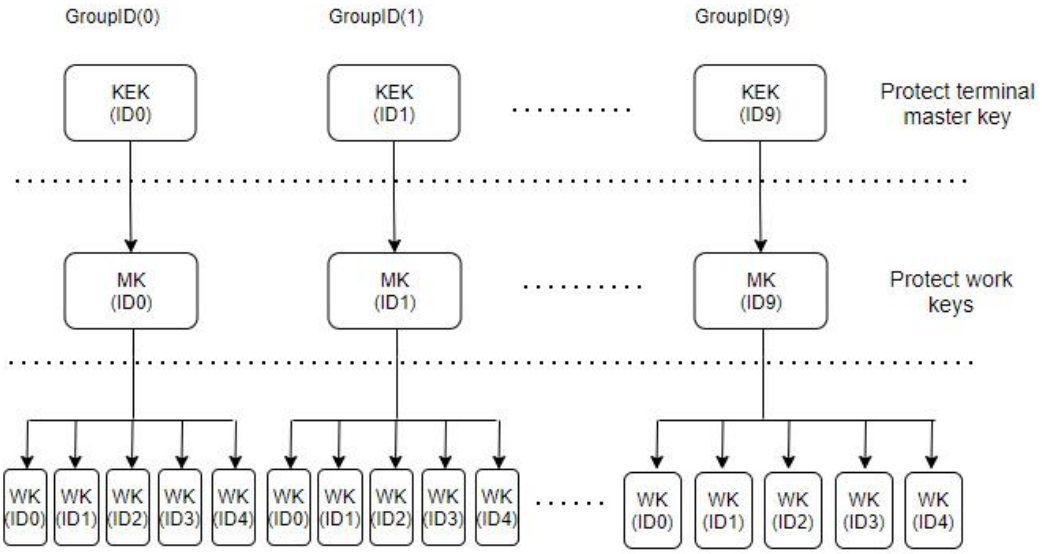
Module Name	Function Name
Display Module	Lib_Lcd
Files System	Lib_File
IC Card Module	Lib_Icc
Magnetic Card Module	Lib_Msc
NFC Module	Lib_Picc
Pinpad	Lib_Pci
Print	Lib_Prn
GPRS	Lib_Wls
WIFI	Lib_Wifi
Touch Screen	Lib_Tp
Code Scan	Lib_Cam
Network Communication	Lib_Net

CONFIDENTIAL

5 Key System

5.1 MK/SK Key System

In order to guarantee secure transaction service, a MK/SK key management system is applied on the device:



The MK/SK key management system consists of ten sets of keys. Each key set consists of one KEK, one Master Key and five Working Keys. The KEK is used to encrypt Master Key .The Master Key is used to encrypt Working Keys. Working Keys are used to encrypt transaction-related data.

The device supports TR-31 and TR-34 key injection.

The following is a detailed description of each key:

① Key Encrypting Key(KEK):

This KEK is used to protect the device master key injection.

The index range for KEK is from 0 to 9.

② master key(MK):

This master key is used to protect the injection and update of the device work keys.

The index range for master key is from 0 to 9.

MK index 0, is protected and updated by KEK ID0;

MK index 1, is protected and updated by KEK ID1;

MK index 2, is protected and updated by KEK ID2;

MK index 3, is protected and updated by KEK ID3;
MK index 4, is protected and updated by KEK ID4;
MK index 5, is protected and updated by KEK ID5;
MK index 6, is protected and updated by KEK ID6;
MK index 7, is protected and updated by KEK ID7;
MK index 8, is protected and updated by KEK ID8;
MK index 9, is protected and updated by KEK ID9.

③ working key(WK):

WK includes encrypting PIN key, MAC key and Track key, etc..

The initial device work key can be injected in a secure environment, or updated through the relevant API interface for updating the key.

Every five working keys are protected by one master key.

The index range for the working keys of each group master is from 0 to 4.

WK group 0 index: 0~4, is protected and updated by master key ID0;
WK group 1 index: 0~4, is protected and updated by master key ID1;
WK group 3 index: 0~4, is protected and updated by master key ID2;
WK group 4 index: 0~4, is protected and updated by master key ID3;
WK group 5 index: 0~4, is protected and updated by master key ID4;
WK group 6 index: 0~4, is protected and updated by master key ID5;
WK group 7 index: 0~4, is protected and updated by master key ID6;
WK group 8 index: 0~4, is protected and updated by master key ID7;
WK group 9 index: 0~4, is protected and updated by master key ID8;
WK group 10 index: 0~4, is protected and updated by master key ID9.

5.2 DUPKT Key System

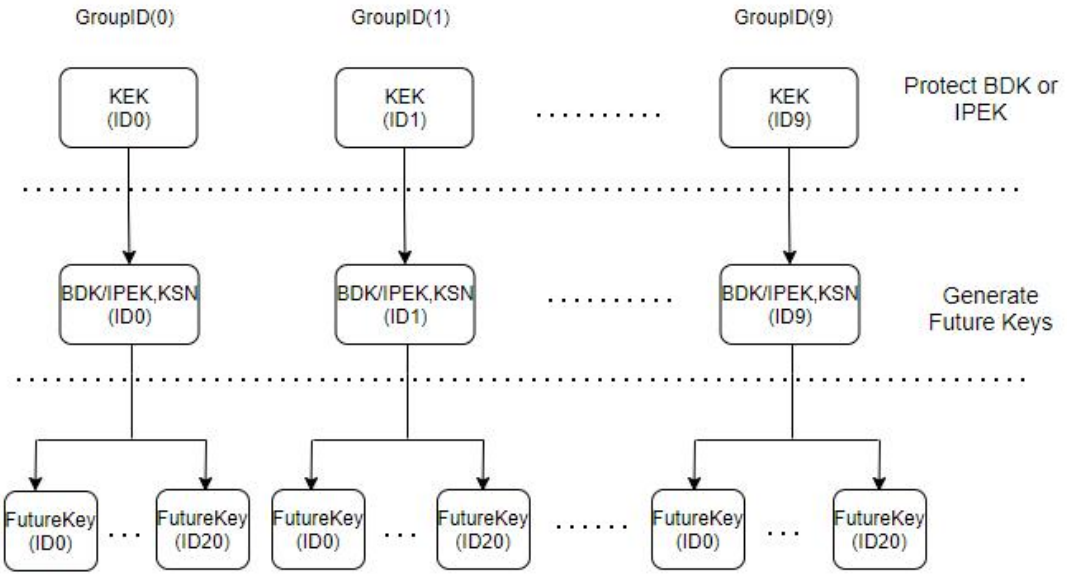
The DUKPT key management system consists of ten sets of keys. Each key set consists of one KEK, and one set of DUKPT Keys. The KEK is used to encrypt BDK or IPEK.

Inject BDK or IPEK and KSN into the device and it will automatically generate 21 sets of future keys . Each set of future keys will generate 5 sets of internal keys. Select different keys to perform encryption and decryption operations by specifying the key group and ID parameters in the key group.

The device supports TR-31 and TR-34 key injection.

CONFIDENTIAL

DUKPT



The following is a detailed description of each key:

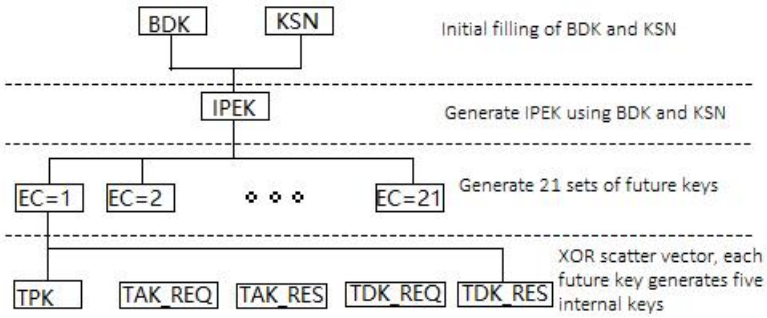
④ Key Encrypting Key(KEK):

This KEK is used to protect BDK or IPEK.

The index range for KEK is from 0 to 9.

⑤ BDK, IPEK and KSN:

The device will automatically generate future keys according to BDK or IPEK and KSN.



For interfaces related to the DUKPT key system, refer to the DUKPT-related interfaces and the demo.

6 TMS Upgrade

The SDK supports TMS upgrade, which enables the remote firmware update of the device. For the related interfaces, refer to the TMS upgrade interface descriptions in this document and the demo.

CONFIDENTIAL

7 API Description

7.1 System Interface

7.1.1 Lib_OsApiInit

Prototype	**void Lib_OsApiInit(void *os_api);**	
Function	Initializes system startup parameters.Initializes the system startup parameters using the configuration passed by the startup code. This function is invoked during the system startup phase, before the main function is entered.	
Parameters	**os_api[in]**	Pointer to the system startup parameter structure, provided by the startup code.
Return	N/A	
Remark	1.This function is called by the system startup code before the main function is entered. Application code shall not call it. 2.The structure pointed to by os_api is provided by the startup code and shall not be modified by the application. 3.The calling position of this function shall not be changed. Refer to the demo for the correct calling method.	

7.1.2 Lib_SysInit

Prototype	**int Lib_SysInit(void);**	
Function	Initializes the system.	
Parameters	No input parameters.	
Return	0	success
	<0	fail
Remark	The calling position of this function shall not be changed. Refer to the demo for the correct calling method.	

7.1.3 Lib_AppInit

Prototype	**int Lib_AppInit(void);**	
Function	application initialization	
Parameters	No input parameters.	
Return	0	initialization succeed
	<0	initialization fail
Remark	The calling position of this function shall not be changed. Refer to the demo for the correct calling method.	
Example	*//app initialization* *void main(void)* *{*	

CONFIDENTIAL

	int iret; *iret=Lib_AppInit();* *If(!iret)* *{* *//app parameters initialization；* *}* *}*

7.1.4 Lib_DelayMs

Prototype	**void Lib_DelayMs(unsigned long ulMs);**	
Function	Delays execution for a specified time.	
Parameters	**ulMs[in]**	delay time (unit: milliseconds)
Return	N/A	
Remark		

7.1.5 Lib_RtcToUtcSec

Prototype	**int Lib_RtcToUtcSec(unsigned char *pucTimeBuf, unsigned char ucTimeBufSize, unsigned long *pulUtc);**	
Description	Converts an RTC time value to a UTC time value expressed in seconds. The input time is provided as a 6-byte BCD buffer in the format YYMMDDhhmmss (year, month, day, hour, minute, second).	
Parameters	**pucTimeBuf[in]**	Pointer to a 6-byte buffer containing the RTC time in BCD format (YYMMDDhhmmss).
	ucTimeBufSize[in]	Size of the time buffer. It shall be 6.
	pulUtc[out]	Pointer to a variable that receives the converted UTC time in seconds.
Return	0	success
	<0	fail
Remark		

7.1.6 Lib_UtcSecToRtc

Prototype	**int Lib_UtcSecToRtc(unsigned long ulUtc, unsigned char *pucTimeBuf, unsigned char ucTimeBufSize);**
Description	Converts a UTC time value expressed in seconds to an RTC time value. The output time is written to a 6-byte BCD buffer in the format YYMMDDhhmmss (year, month, day, hour, minute,

CONFIDENTIAL

	second).	
Parameters	**ulUtc[in]**	UTC time in seconds to be converted.
	pucTimeBuf[out]	Pointer to a buffer that receives the RTC time in BCD format (YYMMDDhhmmss). The buffer shall be at least 6 bytes.
	ucTimeBufSize[in]	Size of the time buffer. It shall be 6.
Return	0	success
	<0	fail
Remark		

7.1.7 Lib_SetDateTime

Prototype	**int Lib_SetDateTime(unsigned char *pucBuf);**	
Function	Sets the system date and time.	
Parameters	**pucBuf[in]**	Sets the system date and time. The time is provided as a 6-byte BCD buffer in the format YYMMDDhhmmss (year, month, day, hour, minute, second).
Return	0	success
	<0	fail
Remark	pucBuf shall point to a valid 6-byte buffer in BCD format; it shall not be NULL.	
Example	*uchar DateTimeStr[7];* *memcpy (DateTimeStr, "\x08\x01\x21\x09\x56\x42", 6);* *Lib_SetDateTime(DateTimeStr);* *// Change the system time to 9:56:42 on January 21, 2008*	

7.1.8 Lib_GetDateTime

Prototype	**int Lib_GetDateTime(unsigned char *pucBuf);**	
Function	Gets the system date and time.	
Parameters	**pucBuf[out]**	Pointer to a buffer that receives the date and time in BCD format (YYMMDDhhmmss). The buffer shall be at least 6 bytes.
Return	0	success
	<0	fail
Remark		

7.1.9 Lib_SyncDateTime

Prototype	**int Lib_SyncDateTime(char *sntp, int timeZone, int**

CONFIDENTIAL

	timeoutMs);	
Description	Synchronizes the system time from an NTP server.	
Parameters	sntp[in]	NTP server address, specified as a domain name or an IP address.
	timezone[in]	Time zone offset, in units of 15 minutes. For example, UTC+8 is represented as 32 (8 × 60 ÷ 15).
	timeoutMs [in]	Synchronization timeout, in milliseconds.
Return	0	success
	<0	fail
Remark	1.This function requires network connectivity. Ensure that the network is available before calling it. 2.sntp shall point to a valid NTP server address string; it shall not be NULL. 3.This function blocks until the synchronization completes or the specified timeout expires.	

7.1.10 Lib_TimeGetUnixTime

Prototype	unsigned long Lib_TimeGetUnixTime(void);	
Description	Gets the current Unix time.Returns the current Unix time, that is, the number of seconds elapsed since 00:00:00 UTC on 1 January 1970.	
Parameters	No input parameters.	
Return	>0	Current Unix time in seconds.
	0	fail
Remark		

7.1.11 Lib_SysGetTimer

Prototype	unsigned long Lib_SysGetTimer(void);
Function	Returns the current value of the system timer in milliseconds. This function is used to measure elapsed time by calculating the difference between two calls.
Parameters	No input parameters.
Return	Current system timer value, in milliseconds.
Remark	The returned value is intended for measuring elapsed time, that is, by calculating the difference between two calls. It is not a calendar time value.

CONFIDENTIAL

7.1.12 Lib_ReadSn

Prototype	**int Lib_ReadSn(unsigned char *snBuf, unsigned long bufSize);**	
Function	Reads the device serial number.	
Parameters	**snBuf[out]**	Pointer to a buffer that receives the device serial number.
	bufSize[in]	Size of the buffer pointed to by snBuf. It shall not be less than 32.
Return	>0	Success. The value is the length of the serial number.
	<=0	fail
Remark		

7.1.13 Lib_GetPN

Prototype	**int Lib_GetPN(unsigned char *pucBuf);**	
Description	Gets the device part number.	
Parameters	**pucBuf[out]**	Pointer to a buffer that receives the device part number. The buffer shall be at least 32 bytes.
Return	0	success
	<0	fail
Remark		

7.1.14 Lib_GetDevState

Prototype	**unsigned char Lib_GetDevState(E_DEV_STATE_TYPE devStateType);**	
Description	get device status	
Parameters	**devStateType[in]**	Device state type to query. See the definition of E_DEV_STATE_TYPE for the available values.
Return	Current device state corresponding to the specified state type.	
Remark	Refer to E_DEV_STATE_TYPE definition. It is mainly used for device status check. It is recommended to refer to the demo code for use. Mainly used for device status check, for implementation, please refer to the demo code.	

7.1.15 Lib_Reboot

CONFIDENTIAL

Prototype	**int Lib_Reboot(void);**
Description	Reboots the device. The system restarts after this function is called.
Parameters	No input parameters.
Return	N/A
Remark	Before calling this function, the application shall complete pending operations and save the data that needs to be retained.

7.1.16 Lib_ShutDown

Prototype	**int Lib_ShutDown(void);**
Description	Shuts down the device. The system powers off after this function is called.
Parameters	No input parameters.
Return	N/A
Remark	Before calling this function, the application shall complete pending operations and save the data that needs to be retained.

7.1.17 Lib_SysSleep

Prototype	**void Lib_SysSleep(void);**
Description	Puts the device into sleep mode. In sleep mode, the device enters a low-power state and waits for a wakeup source.
Parameters	No input parameters.
Return	N/A
Remark	

7.1.18 Lib_FactoryRecovery

Prototype	**int Lib_FactoryRecovery(void);**	
Description	Deletes the device parameters. The factory defaults take effect after the device is restarted.	
Parameters	No input parameters.	
Return	0	success
	<0	fail
Remark	1.This function does not reboot the device. The device remains in its current running state after the parameters are deleted.	

CONFIDENTIAL

	2.The factory defaults take effect only after the device is restarted. If the defaults are to be applied immediately, the application shall reboot the device by calling Lib_Reboot after this function returns success. 3.The device parameters are permanently deleted by this function. The application shall save any data that needs to be retained before calling it.

7.1.19 Lib_ExtPower

Prototype	**int Lib_ExtPower(void);**	
Description	Gets the status of the external power adapter.	
Parameters	No input parameters.	
Return	0x0	The external power adapter is unplugged.
	non-zero	The external power adapter is plugged in.
Remark		

7.1.20 Lib_PowerChargingCtrl

Prototype	**void Lib_PowerChargingCtrl(void)**
Description	Enters the off-state charging mode.
Parameters	**No input parameters.**
Return	N/A
Remark	

7.1.21 Lib_GetPwrKeyStatus

Prototype：	**u8 Lib_GetPwrKeyStatus(void);**	
Description	Gets the power key status.	
Parameters	No input parameters.	
Return：	0	The power key is not pressed.
	1	The power key is pressed.
Remark：		

7.1.22 Lib_GetPowerOnCause

Prototype	**unsigned long Lib_GetPowerOnCause(void);**

CONFIDENTIAL

Description	Gets the cause of the last device power-on.
Parameters	No input parameters.
Return	Power-on cause. Refer to the definition of E_POWERON_CAUSE for the available values.
Remark	

7.1.23 Lib_MaintenanceMode

Prototype	**void Lib_MaintenanceMode(void);**
Description	Makes the device enter the maintenance mode. The device stays in this mode until the user exits it by pressing the cancel key.
Parameters	No input parameters.
Return	N/A
Remark	1.After this function is called, the device stays in the maintenance mode and does not return to the normal mode automatically. 2.The maintenance mode is exited by pressing the cancel key.

7.1.24 Lib_WDGReloadCounter

Prototype	**void Lib_WDGReloadCounter(void);**
Description	Reloads the watchdog counter, which resets the watchdog timer and prevents the device from being reset by the watchdog.
Parameters	No input parameters.
Return	N/A
Remark	

7.1.25 Lib_TamperIsOccured

Prototype	**int Lib_TamperIsOccured(unsigned char *pucType);**	
Function	get security status	
Parameters	**pucType[out]**	type of triggered security status
Return	0	security status triggered
	<0	secure, without any security status triggered
Remark	After a security-tamper status is triggered, read the type of triggered security status.	

CONFIDENTIAL

7.1.26 Lib_KbGetKey

Prototype	**unsigned char Lib_KbGetKey(void);**
Function	key detection
Parameters	No input parameters.
Return	key value
Remark	

7.1.27 Lib_Beep

Prototype	**void Lib_Beep(unsigned long ulMs);**	
Function	Beep according to the configured duration	
Parameters	**ulMs[in]**	Buzzer sounding duration in milliseconds.
Return	N/A	

Remark	*example：*
	Lib_Beep(25); //sound a 25ms beep
	Lib_Beep(50); //sound a 50ms beep	

7.1.28 Lib_SdkCallbackInit

Prototype	**void Lib_SdkCallbackInit(PST_SDK_CALLBACK pCallback);**	
Description	Registers the callback functions with the SDK. The SDK invokes these callbacks when the corresponding events occur.	
Parameters	**pCallback[in]**	Pointer to the SDK callback structure, which contains the callback function pointers to be registered.
Return	N/A	
Remark	1.The structure pointed to by pCallback shall be initialized by the caller before it is passed to this function. Refer to the definition of ST_SDK_CALLBACK for the available callback functions. 2.The structure pointed to by pCallback shall remain valid after this function returns, because the SDK holds a reference to it. 3.The calling position of this function shall not be changed. Refer to the demo for the correct calling method.	

7.1.29 Lib_PrintfHeapInfo

Prototype	**int Lib_PrintfHeapInfo(void);**	
Function	Prints the heap information.	
Parameters	No input parameters.	
Return	0	success

	<0	fail
Remark	The heap information is output to the log system.	

7.1.30 Lib_malloc

Prototype	**void *Lib_malloc(unsigned long ulSize);**	
Description	Allocates a block of memory of the specified size and returns a pointer to it.	
Parameters	**ulSize[in]**	Size of the memory block to allocate, in bytes.
Return	Non-NULL	Memory allocated successfully; the pointer to the allocated memory block is returned.
	NULL	Memory allocation failed, for example, when there is not enough memory available.
Remark	1.The return value shall be checked before use. The application shall not dereference a NULL pointer. 2.The allocated memory is not initialized. The application shall initialize it before use if required. 3.The memory block returned by this function shall be released by Lib_free when it is no longer required; otherwise, a memory leak will occur. 4.The application shall use Lib_malloc and Lib_free for memory allocation and release. Do not mix them with the standard library functions malloc and free.	

7.1.31 Lib_free

Prototype	**void Lib_free(void *ptr);**	
Description	Releases a block of memory.	
Parameters	**ptr[in]**	Pointer to the memory block to be released. It shall be a pointer returned by Lib_malloc.
Return	N/A	
Remark	1.ptr shall be a pointer returned by Lib_malloc. Do not pass a pointer that was not allocated by Lib_malloc, and do not pass a pointer to a stack or static variable. 2.Do not release the same memory block more than once. 3.The application shall use Lib_malloc and Lib_free for memory allocation and release. Do not mix them with the standard library functions malloc and free.	

7.1.32 Lib_GetHardState

Prototype：	**unsigned long Lib_GetHardState(void);**

CONFIDENTIAL

Description	Gets the hardware status.	
Parameters	No input parameters.	
Return	0	The hardware status is normal.
	non-zero	The hardware status is abnormal. See the bit definitions below. Bit 0: Security chip fault. 0: normal, 1: fault. Other bits: Reserved.
Remark	1.If the hardware status is abnormal, the device will enter maintenance mode. 2.The return value is a bit field. The application shall check bit 0 to determine whether the security chip is faulty.	

7.1.33 Lib_CheckUpgradeFlag

Prototype	**unsigned char Lib_CheckUpgradeFlag(void);**	
Description	Checks the device upgrade result.	
Parameters	No input parameters.	
Return：	0	No upgrade has been performed.
	1	The upgrade succeeded.
Remark：		

7.1.34 Lib_SetInfo

Prototype	**int Lib_SetInfo(E_INFO ucId, void *pvInfoBuf, unsigned long ulInfoLen);**	
Description	Sets the device information.	
Parameters	**ucId[in]**	Device information type to set. Refer to the definition of E_INFO for the available values.
	pvInfoBuf [in]	Pointer to the buffer containing the data to be set.
	ulInfoLen [in]	Length of the data pointed to by pvInfoBuf.
Return	0	success
	<0	fail
Remark	Device parameter types refer to the following type definitions: typedef enum { E_FONT_NAME = 0, // font file name	

CONFIDENTIAL

	E_FONT_SIZE, // font file size E_LANG_TYPE, // language type E_MENU_XML_NAME, // XML menu filename E_CHARACTER_ENCODING_TYPE, // Character encoding type Please refer to the type to pass the value: E_CHARACTER_ENCODING_COLLECTION }E_INFO; Notice: 1. When setting the E_FONT_SIZE and E_CHARACTER_ENCODING_TYPE parameters, please pass in the unsigned long type pointer, and the parameter length is 4; 2. When setting the E_LANG_TYPE parameter, please pass in the unsigned char type pointer, and the parameter length is 1; The above usage can refer to the DEMO program usage.

7.1.35 Lib_GetInfo

Prototype	**int Lib_GetInfo(E_INFO ucId, void *pvInfoBuf, unsigned long *pulInfoLen);**	
Description	Gets the device information.	
Parameters	**ucId[in]**	Device information type to get. Refer to the definition of E_INFO for the available values.
	pvInfoBuf[in]	Pointer to the buffer that receives the device information. The buffer shall be allocated by the caller.
	pulInfoLen[in/out]	Input: size of the buffer pointed to by pvInfoBuf. Output: actual length of the data written.
Return	0	success
	<0	fail
Remark	1.Refer to the demo for the correct usage of this function. 2.ucId shall be a value defined in E_INFO. 3.pvInfoBuf and pulInfoLen shall not be NULL.	

7.1.36 Lib_XmlCvtIdToStr

Prototype	**const char* Lib_XmlCvtIdToStr(unsigned short usId);**	
Description	Gets the string corresponding to an ID from the XML file.	
Parameters	**usId[in]**	string ID defined in XML file
Return	“Error”	no string matching this ID was found in the XML file
	other strings	the string corresponding to this ID
Remark		

CONFIDENTIAL

7.1.37 Lib_Trace

Prototype：	**void Lib_Trace(const char *fmt, ...);**	
Description	Outputs a formatted log message.	
Parameters	**fmt ...**	Format string, followed by a variable number of arguments.
Return	N/A	
Remark	For example： Lib_Trace("System Init\r\n"); Lib_Trace("BatVolt=%d\r\n", Volt); 1.The maximum length of a single log message is 1000 bytes. Data exceeding this length is truncated. 2.This function shall be used instead of the standard printf function so that the log output is managed by the SDK.	

7.1.38 Lib_TraceBuf

Prototype	**void Lib_TraceBuf(unsigned char *pucBuf, unsigned short usLen);**	
Description	Outputs a data buffer as a log message.	
Parameters	**pucBuf [in]**	Pointer to the data buffer to be output.
	usLen [in]	Length of the data to be output.
Return：	N/A	
Remark：	unsigned char aucBuf[10]; Lib_TraceBuf(aucBuf, sizeof(aucBuf));	

7.1.39 Lib_TraceLogConfig

Prototype	**int Lib_TraceLogConfig(unsigned char ucFlag);**	
Description	Configures the log output mode.	
Parameters	**ucFlag [in]**	0x00-Turns off log output. 0x01-Turns on log output. 0x02-Turns on log output and saves the log to a file. 0x03-Uploads the log file to the server. 0x80-Turns on CP log output.
Return	0	success

CONFIDENTIAL

	<0	fail
Remark	1.This function affects the output of Lib_Trace and Lib_TraceBuf.	

7.1.40 Lib_GetVersion

Prototype	**int Lib_GetVersion(unsigned char *pucBuf, int buf_size, unsigned char flag);**	
Description	Gets the version of the specified module.	
Parameters	**pucBuf [out]**	Pointer to the buffer that receives the version string.
	ulBufSize[in]	Maximum length of the buffer pointed to by pucBuf.
	ucFlag[in]	Module selector. See the table below for the available values. 0 - OS version 1 - SDK version 2 - SE version
Return	0	success
	<0	fail
Remark		

7.1.41 Lib_realloc

Prototype	**void *Lib_realloc(void *ptr, unsigned long size);**	
Description	Reallocates a memory block previously allocated by Lib_malloc, changing its size to size bytes. The contents are preserved up to the smaller of the old and new sizes.	
Parameters	**ptr[in]**	Pointer to a previously allocated memory block. If NULL, a new block is allocated.
	size[in]	New size of the memory block, in bytes.
Return	Non-NULL	Success. Returns a pointer to the reallocated memory block. This pointer can be freed with Lib_free.
	NULL	Failure. The original memory block ptr remains unchanged and is not freed.
Remark	1.ptr = Lib_realloc(ptr, size); directly. If allocation fails, ptr becomes NULL and the original block is lost, causing a memory leak. Always store the return value in a temporary pointer first. 2.On failure, the original memory block remains valid. Decide whether to keep using it or free it with Lib_free. 3.This function can only be called with pointers returned by Lib_malloc. Do not pass pointers to stack variables, static	

CONFIDENTIAL

	variables, or memory from other pools. 4.If Lib_realloc succeeds, the original pointer ptr becomes invalid immediately. Do not use or free it afterwards.

7.2 Algorithm

7.2.1 Lib_AlgDes

Prototype	**unsigned char Lib_AlgDes(unsigned char *pucVector, unsigned char *pucKey, unsigned char *pucData, unsigned long dwSize, unsigned char ucMode);**	
Function	Des algorithm operation	
Parameters	**pucVector[in]**	vector initialization
	pucKey[in]	key
	pucData[in] /[out]	data to be encrypted or decrypted
	dwSize[in]	length of data to be encrypted or decrypted
	ucMode[in]	Encryption and decryption mode： Bit0~Bit1: 0x00: SINGLE KEY 0x01: DOUBLE KEY 0x02: TRIPLE KEY Bit2: 0x00: encryption mode 0x04: decryption mode Bit3: 0x00: ECB mode 0x08: CBC mode
Return	1 - success, 0 - fail	
Remark		

7.2.2 Lib_AlgAes

Prototype	**unsigned char Lib_AlgAes(unsigned char *pucVector, unsigned char *pucKey, unsigned char *pucData, unsigned long dwSize, unsigned char ucMode);**	
Function	AES algorithm	
Parameters	**pucVector[in]**	vector initialization
	pucKey[in]	key

	pucData[in]/ [out]	data to be encrypted or decrypted
	dwSize[in]	length of data to be encrypted or decrypted
	ucMode[in]	Encryption and decryption mode： Bit0~Bit1: 0x00: AES_KEYLEN_128 0x01: AES_KEYLEN_192 0x02: AES_KEYLEN_256 Bit2: 0x00: encryption mode 0x04: decryption mode Bit3: 0x00: ECB mode 0x08: CBC mode
Return	1 - success, 0 - fail	
Remark		

7.2.3 Lib_AlgHash(SHA-1)

Prototype	**void Lib_AlgHash(unsigned char *pucDataIn, unsigned long ulDataInLen, unsigned char *pucDataOut);**	
Function	for any length of input data, output 20-byte HASH result	
Parameters	**pucDataIn[in]**	input data buffer pointer
	ulDataInLen[in]	input data length (in bytes)
	pucDataOut[out]	output data buffer pointer
Return	N/A	
Remark	this function uses the secure HASH algorithm SHA-1 (according to ISO / IEC 10118-3 or FIPS 180-1)	

7.2.4 Lib_AlgHash256

Prototype	**void Lib_AlgHash256(unsigned char *pucDataIn, unsigned long ulDataInLen, unsigned char *pucDataOut);**	
Function	HASH256 algorithm (SHA-256 algorithm)	
Parameters	**pucDataIn[in]**	data buffer to be encrypted
	ulDataInLen[in]	length of the data to be encrypted
	pucDataOut[out]	encrypted data buffer
Return	N/A	
Remark		

CONFIDENTIAL

7.2.5 Lib_AlgAes256gcmEncrypt

Prototype	**int Lib_AlgAes256gcmEncrypt(const unsigned char *key256, unsigned int data_length, const unsigned char *iv, unsigned int iv_len, const unsigned char *aad, unsigned int aad_len, const unsigned char *input, unsigned char *output, unsigned int tag_len, unsigned char *tag);**	
Function	AES-256-GCM Encrypt	
Parameters	**key256[in]**	256-bit key (32 bytes), provided by user for encryption
	data_length [in]	Data length in bytes.
	iv[in]	Initialization Vector (Nonce), recommended 12 bytes (96 bits), must be unique for each encryption with the same key.
	iv_len[in]	Length of IV in bytes (usually 12).
	aad[in]	Additional Authenticated Data, can be NULL. This data is not encrypted but is authenticated; integrity is verified during decryption.
	aad_len[in]	Length of AAD in bytes; pass 0 if aad is NULL.
	input[in]	Plaintext data (original bytes to be encrypted).
	output[out]	Output ciphertext buffer, must be at least data_length bytes (ciphertext length equals plaintext).
	tag_len[in]	Desired authentication tag length in bytes, typically 16 (128 bits), also 12 or 8 allowed.
	tag[out]	Output authentication tag buffer, at least tag_len bytes; the tag is written here after encryption.
Return	0x0	success
	non-zero	fail
Remark		

7.2.6 Lib_AlgAes256gcmDecrypt

Prototype	**int Lib_AlgAes256gcmDecrypt(const unsigned char *key256, unsigned int data_length, const unsigned char *iv, unsigned int iv_len, const unsigned char *aad, unsigned int aad_len, const unsigned char *tag, unsigned int tag_len, const unsigned char *input, unsigned char *output);**	
Function	AES-256-GCM Decrypt	
Parameters	**key256[in]**	256-bit key (32 bytes), provided by user for encryption
	data_length [in]	Data length in bytes.

CONFIDENTIAL

	iv[in]	Initialization Vector (Nonce), recommended 12 bytes (96 bits), must be unique for each encryption with the same key.
	iv_len[in]	Length of IV in bytes (usually 12).
	aad[in]	Additional Authenticated Data, can be NULL. This data is not encrypted but is authenticated; integrity is verified during decryption.
	aad_len[in]	Length of AAD in bytes; pass 0 if aad is NULL.
	tag[in]	Input authentication tag (generated by the encryptor and transmitted), used to verify data integrity.
	tag_len[in]	Length of input tag in bytes, must match the length used during encryption.
	input[in]	Ciphertext data (bytes to be decrypted).
	output[out]	Output plaintext buffer, at least data_length bytes; the plaintext is written here after decryption.
Return	0x0	success
	non-zero	fail
Remark		

7.2.7 Lib_AlgMD5

Prototype	**int Lib_AlgMD5(unsigned char *pucDataIn, unsigned long ulDataInLen, unsigned char *pucDataOut);**	
Function	MD5 algorithm	
Parameters	**pucDataIn[in]**	data buffer to be encrypted
	ulDataInLen[in]	length of the data to be encrypted
	pucDataOut[out]	encrypted data buffer
Return	0x0	success
	non-zero	fail
Remark		

7.2.8 Lib_AlgGetRandom

Prototype：	**void Lib_AlgGetRandom(unsigned char *pucOut, unsigned long ulLen);**	
Description	get random number	
Parameters	**pucOut[out]**	store random number buffer pointer
	ulLen[in]	the length of the random number to be obtained
Return：	N/A	
Remark：		

CONFIDENTIAL

7.2.9 Lib_AlgRsa

Prototype	**int Lib_AlgRsa(unsigned char *pucKey, unsigned long ulKeyLen, unsigned short usProtoKeyBit, unsigned char *pucDataIn, unsigned char *pucDataOut, unsigned char ucMode);**	
Function	RSA encryption and decryption operation	
Parameters	**pucKey[in]**	pointer to the key buffer that store RSA
	ulKeyLen[in]	Key length (in byte)
	usProtoKeyBit[in]	length of modulus
	pucDataIn[in]	input data buffer pointer
	pucDataOut[out]	output data buffer pointer
	ucMode[in]	0x00:encryption 0x04:decryption
Return	0x0	success
	non-zero	fail
Remark		

7.2.10 Lib_AlgRsaKeyPairGen

Prototype	**int Lib_AlgRsaKeyPairGen(unsigned char *pucRsaPk, unsigned char *pucRsaSk, unsigned short usProtoKeyBit);**	
Function	generate public-private key pairs	
Parameters	**pucRsaPk[out]**	pointer to the public key buffer
	pucRsaSk[out]	pointer to the private key buffer
	usProtoKeyBit[int]	length of modulus, 512, 1024 or 2048
Return	0x0	success
	non-zero	fail
Remark		

7.2.11 Lib_AlgRsaKeyPairVerify

Prototype	**int Lib_AlgRsaKeyPairVerify(unsigned char *pucRsaPk, unsigned char *pucRsaSk);**	
Function	verify public-private key pairs	
Parameters	**pucRsaPk[in]**	pointer to the public key buffer
	pucRsaSk[in]	pointer to the private key buffer
Return	0x0	success
	non-zero	fail
Remark		

CONFIDENTIAL

7.2.12 Lib_AlgBase64Encode

Prototype	**int Lib_AlgBase64Encode(unsigned char *dst, unsigned int dlen, unsigned int *olen, const unsigned char *src, unsigned int slen);**	
Function	Base64 Encode data	
Parameters	**dst**[out]	Output buffer to store Base64 encoded result
	dlen[in]	Byte length of output buffer
	olen[out]	returns actual byte count after encoding
	src[in]	Input buffer of original raw data
	slen[in]	Byte length of input raw data
Return	0x0	success
	non-zero	fail
Remark		

7.2.13 Lib_AlgBase64Decode

Prototype	**int Lib_AlgBase64Decode(unsigned char *dst, unsigned int dlen, unsigned int *olen, const unsigned char *src, unsigned int slen);**	
Function	Base64 Decode data	
Parameters	**dst**[out]	Output buffer to store binary data after Base64 decoding
	dlen[in]	Byte length of output buffer
	olen[out]	returns actual binary byte count after decoding
	src[in]	Input buffer containing Base64 encoded string data
	slen[out]	Byte length of input Base64 encoded data
Return	0x0	success
	non-zero	fail
Remark		

7.2.14 Lib_AlgGetCrc16

Prototype	**unsigned short Lib_AlgGetCrc16(unsigned char *pucBuf, unsigned long ulLen);**	
Function	Calculate the CRC16 checksum of input data	
Parameters	**pucBuf**[out]	Data buffer for CRC16 checksum calculation
	ulLen[in]	Byte length of the input data buffer
Return	0x0	success
	non-zero	fail
Remark		

CONFIDENTIAL

7.3 LCD Drive Interface

7.3.1 Lib_LcdSetBrightLevel

Prototype	**void Lib_LcdSetBrightLevel(unsigned char ucLevel);**	
Function	set brightness of display screen	
Parameters	**ucLevel[in]**	1-8, default 5
Return	N/A	
Remark		

7.3.2 Lib_LcdClrScreen

Prototype	**void Lib_LcdClrScreen(void);**
Function	clear the screen
Parameters	No input parameters.
Return	N/A
Remark	

7.3.3 Lib_LcdRefresh

Prototype	**void Lib_LcdRefresh(void);**
Function	refresh screen
Parameters	No input parameters.
Return	N/A
Remark	this function must be to called when LCD-related functions are about to be cal

7.3.4 Lib_LcdDispMemcpy

Prototype	**int Lib_LcdDispMemcpy(unsigned short *src, unsigned short x, unsigned short y, unsigned short w, unsigned short h);**	
Function	refresh display memory to specified area	
Parameters	**src[in]**	input data buffer pointer
	x[in] **y[in]**	starting coordinates
	w[in] **h[in]**	width and height of the area to be refreshed
Return	0	display memory refreshing normal

CONFIDENTIAL

	non-zero	display memory refreshing abnormal
Remark		

7.3.5 Lib_LcdParaSet

Prototype	**void Lib_LcdParaSet(unsigned short w, unsigned short h, unsigned short FontTextEn, unsigned short ForegroundColorText, unsigned short BackgroundColorText);**	
Function	set text widget properties	
Parameters	**w[in]** **h[in]**	set the width and height of the text widget
	FontTextEn[in]	set the font size of the text widget
	ForegroundColorText[in], BackgroundColorText[in]	set the foreground and background colors of the text widget
Return	N/A	
Remark		

7.3.6 Lib_LcdParaGet

Prototype	**void Lib_LcdParaGet(unsigned short *w, unsigned short *h, unsigned short *FontTextEn, unsigned short *ForegroundColorText, unsigned short *BackgroundColorText);**	
Function	get text widget properties	
Parameters	**w[in],h[in]**	get the width and height of the text widget
	FontTextEn[in]	get the font size of the text widget
	ForegroundColorText[in], BackgroundColorText[in]	get the foreground and background colors of the text widget
Return	N/A	
Remark		

CONFIDENTIAL

7.4 LED

7.4.1 Lib_LedControl

Prototype	**int Lib_LedControl(LED_ID ucLedNo, unsigned char ucMode);**		
Description	control LED status(on and off)		
Parameters	**ucLedNo[in]**	number of LED(1~4)	
	ucMode[in]	0	off
		1	on
Return	-1	LED number or mode error	
	0	success	
	others	error	
Remark	LED on the front panel		

7.5 Card Drive Interface

7.5.1 Lib_MscOpen

Prototype	**int Lib_MscOpen(void);**	
Function	open MSR head reader	
Parameters	No input parameters.	
Return	0	success
	non-zero	open fail
Remark	The magnetic card data is read in interrupt mode. Once the magnetic card reader is opened, even if the reading function is not called, the magnetic head can read the magnetic card data as long as the card is swiped. Therefore, when the magnetic card reader is not needed, it is best to close the magnetic card reader	

7.5.2 Lib_MscClose

Prototype	**int Lib_MscClose(void);**	
Function	close MSR head reader	
Parameters	No input parameters.	
Return	0	success
	non-zero	close fail

CONFIDENTIAL

Remark	

7.5.3 Lib_MscRead

Prototype	**int Lib_MscRead(PST_MSC_TRACK pstMscTrack1, PST_MSC_TRACK pstMscTrack2, PST_MSC_TRACK pstMscTrack3);**	
Function	read the data of tracks 1, 2 and 3 in the magnetic card buffer	
Parameters	**pstMscTrack 1[out]**	pointer for storing 1 track data *[application layer buffer should be set to 256]*
	pstMscTrack 2[out]	pointer for storing 2 track data *[application layer buffer should be set to 256]*
	pstMscTrack 3[out]	pointer for storing 3 track data *[application layer buffer should be set to 256]*
Return	0	swiping ongoing, track data has not been obtained yet. meanwhile, the function can be called again to obtain track data.
	1	swiping success
	others	swiping fail

Remark	typedef struct MSC_TRACK
	{
	u8 *pucTrackBuf; // track info buffer
	u8 ucTrackLen; // track info length
	u8 ucTrackValid; // track info effective ID
	}ST_MSC_TRACK, *PST_MSC_TRACK;
	
	normally, reading the magnetic card conforms to ISO7811 standard:
	track1 79 bytes
	track2 40 bytes
	track3 107 bytes	

7.5.4 Lib_IccOpen

Prototype	**int Lib_IccOpen(ICC_SLOT iccSlot);**	
Function	open the specified card channel	
Parameters	**iccSlot [in]**	card initialization channels: 0x00 -> (EMV)IC card channel 0x80 -> (non-EMV)IC card channel 0x01 -> SAM1 channel 0x02 -> SAM2 channel
Return	0	initialization success.
	(-2403)	channel number error
Remark		

CONFIDENTIAL

7.5.5 Lib_IccClose

Prototype	**int Lib_IccClose(ICC_SLOT iccSlot);**	
Function	power off the card in the designated deck	
Parameters	**iccSlot [in]**	card channels: 0x00 -> (EMV)IC card channel 0x80 -> (non-EMV)IC card channel 0x01 -> SAM1 channel 0x02 -> SAM2 channel
Return	0	success
	-2403	channel number error
Remark		

7.5.6 Lib_IccInquiry

Prototype	**int Lib_IccInquiry(ICC_SLOT iccSlot, unsigned char *pucAtr, unsigned char *pucAtrLen, unsigned char *pucUid, unsigned char *pucUidLen);**	
Function	card search and return reset information	
Parameters	**iccSlot[in]**	card channels: 0x00 -> (EMV)IC card channel 0x80 -> (non-EMV)IC card channel 0x01 -> SAM1 channel 0x02 -> SAM2 channel
	pucAtr[out]	get ATR info
	pucAtrLen[out]	get ATR info length
	pucUid[out]	get UID info
	pucUidLen[out]	get UID info length
Return	>0	card search success
	0	card search fail
	(-2403)	channel number error
Remark		

7.5.7 Lib_IccCommand

Prototype	**int Lib_IccCommand(ICC_SLOT iccSlot, unsigned char *pucSend, unsigned short uSendLen, unsigned char *pucRecv, unsigned short *puRecvLen);**	
Function	IC card operation function supporting universal interface protocol(T=0 and T=1)	
Parameters	**iccSlot [in]**	card channels: 0x00 -> (EMV)IC card channel 0x80 -> (non-EMV)IC card channel 0x01 -> SAM1 channel 0x02 -> SAM2 channel

CONFIDENTIAL

	pucSend[in]	Cmd data
	uSendLen[in]	Cmd data length
	pucRecv[out]	card Return data
	puRecvLen[out]	card Return data length
Return	>0	execution success
	0	card not responding
	(-2403)	channel selection error
Remark		

7.5.8 Lib_IccIsPresent

Prototype	**int Lib_IccIsPresent(ICC_SLOT ucSlot);**	
Function	check card isPresent status	
Parameters	**ucSlot [in]**	card initialization channels: 0x00 -> (EMV)IC card channel 0x80 -> (non-EMV)IC card channel 0x01 -> SAM1 channel 0x02 -> SAM2 channel
Return	1	card plug-in
	(-2403)	channel number error
	0	no card plug-in
Remark		

7.5.9 Lib_PiccOpen

Prototype	**int Lib_PiccOpen(void);**	
Description	open NFC module	
Parameters	No input parameters.	
Return	0	success
	non-zero	open fail
Remark		

7.5.10 Lib_PiccClose

Prototype	**int Lib_PiccClose(void);**
Description	close NFC module and keep it closed
Parameters	No input parameters.

CONFIDENTIAL

Return	0	success
	non-zero	Close fail
Remark	after calling this function, the NFC module closed, and the module no longer radiates RF carriers. After that, all function calls will fail except Lib_PiccOpen().	

7.5.11 Lib_PiccInquiry

Prototype	**int Lib_PiccInquiry(unsigned char *pucAtr, unsigned char *pucAtrLen, unsigned char *pucUid, unsigned char *pucUidLen);**	
Description	search for PICC cards; select and activate it when found	
Parameters	**pucAtr[out]**	get ATR data
	pucAtrLen[out]	get ATR data length
	pucUid[out]	get UID data
	pucUidLen[out]	get UID data length
Return	>0	success
	0	no card present in the RF field.
	others	fail
Remark		

7.5.12 Lib_PiccCommand

Prototype	**int Lib_PiccCommand(unsigned char *pucSend, unsigned short uSendLen, unsigned char *pucRecv, unsigned short *puRecvLen);**	
Description	send APDU data and receive the response on the specified channel	
Parameters	**pucSend[in]**	Cmd data
	uSendLen[in]	Cmd data length
	pucRecv[out]	card Return data
	puRecvLen[out]	card Return data length
Return	>0	success
	0	present in the RF field.

CONFIDENTIAL

	others	data exchange error

7.5.13 Lib_PiccRemove

Prototype	**int Lib_PiccRemove(void);**	
Description	judge whether the card has been removed from the sensing area	
Parameters	No input parameters.	
Return	0	success, card removed
	(-3525)	card unremoved
Remark		

7.6 Touch Screen Interface

7.6.1 Lib_TpGetCoordinates

Prototype	**int Lib_TpGetCoordinates(unsigned short *pusX, unsigned short *pusY);**	
Description	Get the coordinates of the touch point	
Parameters	**pusX[out]**	output x-axis coordinate
	pusY[out]	output y-axis coordinate
Return	0x00	success
	<0	no touch sensed and no coordinates got
Remark	call the function repeatedly to get coordinates of touch	

7.6.2 Lib_TpGetESignData

Prototype	**int Lib_TpGetESignData(unsigned char *pucOut, char *pscFeatureCode, int iTimeout);**	
Description	get signature data	
Parameters	**pucOut[out]**	signature data
	pscFeature	signature character code

CONFIDENTIAL

	Code[in]	
	iTimeout[in]	signature timeout
Return	0x00	success
	<0	no touch sensed and no coordinates got
Remark	call the function repeatedly to get coordinates of touch	

7.6.3 Lib_TpCalibrate

Prototype	**int Lib_TpCalibrate(void);**	
Description	touch screen calibration	
Parameters	No input parameters.	
Return	0x00	success
	<0	failed
Remark	If the touch screen is misaligned, call this interface to recalibrate it. Note: This interface applies only to resistive touch panels. Capacitive touch panels do not require calibration.	

7.6.4 Lib_TpGetKey

Prototype	**unsigned char Lib_TpGetKey(TP_TOUCH_KEY *keyList, unsigned char keyNum);**	
Description	Get the key value touched	
Parameters	**keyList[in]**	input TP_TOUCH_KEY
	keyNum[in]	input key number
Return	> 0	Key Value
	0	No key was pressed.
Remark	typedef struct _TOUCH_AREA { unsigned short x; unsigned short y; unsigned short w; unsigned short h; } TP_TOUCH_AREA;	

CONFIDENTIAL

7.7 Code Scan Module

7.7.1 Lib_CamInit

Prototype	int Lib_CamInit(void）;	
Description	initialize code scan module	
Parameters	No input parameters.	
Return	0	success
	non-zero	error
Remark		

7.7.2 Lib_CamOpen

Prototype	int Lib_CamOpen(void);	
Description	open code scan module	
Parameters	No input parameters.	
Return	0	success
	non-zero	error
Remark		

7.7.3 Lib_CamClose

Prototype	int Lib_CamClose(void);	
Description	close code scan module	
Parameters	No input parameters.	
Return	0	success
	non-zero	error
Remark		

7.7.4 Lib_CamGetResult

Prototype	int Lib_CamGetResult(unsigned char *pucBuf, unsigned short usMaxLen);

CONFIDENTIAL

Description	get code scan result	
Parameters	**pucBuf[in]**	result buffer
	usMaxLen[in]	result buffer length
Return	>0	length of code scan result
	0	result not obtained
	< 0	error
Remark	non-blocking interface, which can be called repeatedly to get the result	

7.7.5 Lib_CamDisp

Prototype	**void Lib_CamDisp(unsigned char enable);**		
Description	control whether the camera opens the preview image function		
Parameters	**ucMode[in]**	0	off
		1	on
Return	N/A		
Remark			

7.7.6 Lib_CamLed

Prototype	**void Lib_CamLed(unsigned char ucMode);**		
Description	control LED		
Parameters	**ucMode[in]**	0	off
		1	on
Return	N/A		
Remark			

7.8 GPRS Communication Interface

7.8.1 Lib_WlsInit

Prototype	**int Lib_WlsInit(void);**
Description	initialize GPRS module

CONFIDENTIAL

Parameters	No input parameters.	
Return	0	success
	<0	fail
Remark	the system automatically initializes the GPRS module when it is turned on, and it can be called when the module is abnormal during use.	

7.8.2 Lib_WlsGetInitStatus

Prototype	**int Lib_WlsGetInitStatus(void);**	
Description	get GPRS module status	
Parameters	No input parameters.	
Return	0	success
	<0	fail
Remark	this function can be called repeatedly to query initialization status, other functions only can be called after initialization.	

7.8.3 Lib_WlsCheckSignal

Prototype	**int Lib_WlsCheckSignal(int * pSignalLevel);**	
Description	check network signal level	
Parameters	**pSignalLevel[out]**	signal value returned
Return	0	success
	<0	fail
Remark	return success, the signal value is valid	

7.8.4 Lib_WlsCheckSim

Prototype:	**int Lib_WlsCheckSim(void);**	
Description	check sim card status	
Parameters	No input parameters.	
Return	WLS_OK	SIM card is ok to use
	WLS_NOSIM	SIM card does not exist
	0xFF	SIM card status not found

CONFIDENTIAL

Remark	return success, the signal value is valid

7.8.5 Lib_WlsGetInfo

Prototype	**int Lib_WlsGetInfo(E_WLS_INFO_ID id, void *buff, int buffSize)**	
Description	get network related information	
Parameters	**id[in]**	ID to get information
	buff[in]	get info buffer
	buffSize[in]	get the length of the message buffer
Return：	0	success
	others	failed
Remark：		

7.8.6 Lib_WlsGetBSinfo

Prototype：	**int Lib_WlsGetBSinfo(unsigned char *pucBsInfo, unsigned log ulLen);**	
Description	Get base station information	
Parameters	**pucBsInfo[out]**	Get base station information buffer
	ulLen[in]	The length of buffer
Return：	Received data length	success
	<0	error
Remark：		

7.8.7 Lib_WlsSetDns

Prototype	**int Lib_WlsSetDns(unsigned char *pucMainDns, unsigned char *pucBackDns);**	
Description	set DNS	
Parameters	**pucMainDns[in]**	primary DNS
	pucBackDns[in]	backup DNS
Return	0	success

CONFIDENTIAL

	<0	fail
Remark:		

7.8.8 Lib_WlsGetDns

Prototype：	**int Lib_WlsGetDns(char *pscMainDnsBuf, unsigned long ulMainDnsBufLen, char *pcBackDnsBuf, unsigned long ulBackDnsBufLen);**	
Description	Get DNS	
Parameters	**pscMainDnsBuf[out]**	Get main DNS buffer
	ulMainDnsBufLen[in]	The length of buffer
	pscBackDnsBuf[out]	Get back DNS buffer
	ulBackDnsBufLen[in]	The length of buffer
Return：	0	success
	<0	error
Remark：		

7.8.9 Lib_WlsDnsResolve

Prototype	**int Lib_WlsDnsResolve(char *name, char *result, int len);**	
Description	dynamic domain name resolution	
Parameters	**name[in]**	string of DNS or IP address
	result[out]	store parsed string buffer
	len[in]	store the length of the parsed string buffer
Return	0	success
	<0	fail
Remark		

7.8.10 Lib_WlsGetApn

Prototype：	**int Lib_WlsGetApn(char *pscApn, unsigned long**

CONFIDENTIAL

	ulApnLen);	
Description	get APN	
Parameters	**pscApn[out]**	Get APN buffer
	ulApnLen[in]	The length of buffer
Return：	0	success
Remark：	<0	fail
Prototype：		

7.8.11 Lib_WlsSetApn

Prototype	**int Lib_WlsSetApn(char *pscApn, char *pscUserName, char *pscPassword, unsigned char ucType, unsigned char ucAuthProt);**	
Description	set APN	
Parameters	**pscApn[in]**	Access Point Name. Carrier-provided APN string, e.g. cmnet, 3gnet. Must not be NULL.
	pscUsename[in]	Dial-up username. Pass NULL or an empty string for APNs that do not require authentication; required for private/enterprise APNs.
	pscPassword[in]	Dial-up password. Used together with pscUserName; pass NULL or an empty string when authentication is not required.
	ucType[in]	PDP context type. IP protocol version of the bearer.
	ucAuthProt[in]	Authentication protocol. Authentication method used for the PDP context.
Return	0	success
	<0	fail
Remark	need reinitialization after setting	

7.8.12 Lib_WlsSetAdaptiveAPN

Prototype	**int Lib_WlsSetAdaptiveAPN(unsigned char value)**	
Description	APN Adaptive Resident Network	
Parameters	**value[in]**	1: open,0: close
Return：	0	success
	others	failed

CONFIDENTIAL

Remark：	

7.8.13 Lib_WlsGetAPNList

Prototype	XML_APN_NODE_ST* Lib_WlsGetAPNList(unsigned long *ptrApnNum)	
Description	Get APN List	
Parameters	ptrApnNum[out]	input data buffer and output APN number
Return：	0	success
	others	failed
Remark：		

7.8.14 Lib_WlsGetApnAuth

Prototype	int Lib_WlsGetApnAuth(unsigned char *pucType, unsigned char *pucAuthProt)	
Description	Get Current APN Authentication Method And Protocol Type	
Parameters	pucType [out]	Protocol Type(1:ip 2:ipv6 3:ipv4v6)
	pucAuthProt [out]	Prot(0:(NONE) 1:plaintext 2:Cipher)
Return：	0	success
	others	failed
Remark：		

7.8.15 Lib_WlsPing

Prototype	int Lib_WlsPing(char *dstIpStr, long timeout, int size, int pingTimes);	
Description	Ping external network	
Parameters	dstIpStr[in]	Target host IP address string, length must not exceed 15 bytes, e.g. "192.168.1.1".
	timeout[in]	Timeout for each ping, in seconds, range: 1-255.
	size[in]	Test data length, range: 36-1500.
	pingTimes[in]	Number of ping attempts, range: 1-65535.

CONFIDENTIAL

Return	>0	Number of successful pings.
	<=0	fail
Remark		

7.8.16 Lib_WlsGetNetMode

Prototype：	**int Lib_WlsGetNetMode(void);**	
Description	Get network mode	
Parameters	No input parameters.	
Return：	0	automatic(2G/4G)
	2	2G
	4	4G
	other	error
Remark：		

7.8.17 Lib_WlsSetNetMode

Prototype：	**int Lib_WlsSetNetMode(unsigned char netMode);**	
Description	Set network mode	
Parameters	**netMode[in]**	network mode： 0:automatic(2G/4G);2:2G;4:4G
Return：	0	success
	<0	error
Remark：		

7.8.18 Lib_WlsSetAirplaneMode

Prototype：	**int Lib_WlsSetAirplaneMode(unsigned char value);**	
Description	Set airplane mode	
Parameters	**value[in]**	1:indicate enable airplane mode 0:indicate disable airplane mode
Return：	0	success
	<0	error
Remark：		

CONFIDENTIAL

7.8.19 Lib_WlsGetOperatorList

Prototype：	**int Lib_WlsGetOperatorList(char *operatorList, unsigned int length);**	
Description	Get a list of network operators	
Parameters	**operatorList[out]**	network operators receive buffer
	length[in]	Maximum acceptable network operator length
Return：	0	success
	<0	error
Remark：		

7.8.20 Lib_WlsSetOperator

Prototype	**int Lib_WlsSetOperator(unsigned char mode, char *operatorBuf, unsigned int act);**	
Description	set network operator	
Parameters	**operator[in]**	network operator name
	mode[in]	set mode： 0 - automatic mode (operator and network standard are all automatic mode) 1 - Manual Mode
	act[in]	operator name
Return：	0	success
	<0	error
Remark：		

7.8.21 Lib_WlsGetOperator

Prototype	**int Lib_WlsGetOperator(char *operatorBuf, unsigned int length)**	
Description	get current Network operator	
Parameters	**operatorBuf[out]**	buffer which receives current operator information
	length[in]	maximum acceptable operator information character length
Return：	0	success

CONFIDENTIAL

	<0	error
Remark：		

7.8.22 Lib_WlsNetActRefresh

Prototype	**void Lib_WlsNetActRefresh(unsigned char *pucNetAct)**	
Description	Get Current Network Mode	
Parameters	**pucNetAct [out]**	Network Mode
Return：	N/A	
Remark：		

7.9 WiFi Communication Module

7.9.1 Lib_WifiNetworkUp

Prototype	**int Lib_WifiNetworkUp(void);**	
Description	open WiFi communication module	
Parameters	No input parameters.	
Return	0	success
	other	fail
Remark		

7.9.2 Lib_WifiNetworkDown

Prototype	**int Lib_WifiNetworkDown(void);**	
Description	close WiFi communication module	
Parameters	No input parameters.	
Return	0	success
	other	fail
Remark		

CONFIDENTIAL

7.9.3 Lib_WifiNetworkScan

Prototype	**int Lib_WifiNetworkScan(void);**	
Description	WiFi scan AP interface	
Parameters	No input parameters.	
Return	0	success
	other	fail
Remark		

7.9.4 Lib_WifiGetAP

Prototype	**int Lib_WifiGetAP(WIFI_INF *pstWifiInf, unsigned short *pusNum, unsigned short usMax);**	
Description	get AP info	

Parameters	**pstWifiInf [out]**	store the parameters of each AP including:
		typedef struct WIFI_INF
		{
		unsigned char m_ausSSID[50]; // Access Point name
		unsigned char m_aucMode[8]; // Mode: With "Managed" and "auto"
		unsigned char m_aucEncryptType; // Encryption type, "on" with encryption, "off" without encryption
		unsigned char m_aucPass[50] ; // The password used to connect to the AP
		signed long m_aucSignalLevel; // signal strength
		}WIFI_INF;

	pusNum[out]	total number of APS scanned
	usMax[in]	wifi_ max. number of wifi_inf, that is, the maximum number of APS scanned by storage
Return	0	success
	other	fail
Remark		

7.9.5 Lib_WifiConnectAp

Prototype	**int Lib_WifiConnectAp(unsigned char *pucAuthMode, unsigned char *pucEncryptType, char *pscSSID, unsigned char *pucPwd);**

CONFIDENTIAL

Description	WiFi connect to AP interface	
Parameters	**pucAuthMode[in]**	AuthMode Parameters，string,including：OPEN,SHARED,WEPAUTO,WPAPSK,WPA2PSK,WPANONE
	pucEncryptType[in]	EncryptType Parameters，string, including：NONE,WEP,TKIP,AES
	pscSSID[in]	username
	pucPwd[in]	password
Return	0x00	wpa connection success
	0x01	connection success without password
	0x02	wep connection success
	other	fail
Remark	WPA encryption is not supported yet. the name of the wireless network card is ra0	

7.9.6 Lib_WifiDisAp

Prototype	**int Lib_WifiDisAp(void)；**	
Description	disconnect ap	
Parameters	No input parameters.	
Return	0	success
	<0	fail
Remark		

7.9.7 Lib_WifiGetApStatus

Prototype	**int Lib_WifiGetApStatus(void);**	
Description	get the status of the currently connected ap	
Parameters	No input parameters.	
Return	0-STA_AP_CONNECTING	connecting
	1-STA_AP_CONNECTED	connected
	2-STA_AP_DISCONNECTED	disconnect
Remark		

CONFIDENTIAL

7.9.8 Lib_WifiCheckSignal

Prototype	**int Lib_WifiCheckSignal(int * pSignalLevel);**	
Description	check WiFi signal level	
Parameters	**pSignalLevel[out]**	returned signal level
Return	0	success
	<0	fail
Remark	return success，signal value valid	

7.9.9 Lib_WifiSetAutoConnectFlag

Prototype：	**int Lib_WifiSetAutoConnectFlag(bool bFlag);**	
Description	Set up to connect WiFi automatically	
Parameters	**bFlag**	True：connected automatically False：not connected automatically
Return：	0	success
	<0	fail
Remark：		

7.9.10 Lib_WifiGetLinkAp

Prototype：	**int Lib_WifiGetLinkAp(unsigned char* paraData, unsigned short *paraLen, WIFI_PARA_TYPE paraType);**	
Description	Get WiFi information	
Parameters	**paraData[out]**	Get WiFi parameter buffer
	paraLen[in]	Get the length of WiFi parameter buffer
、	**paraType[in]**	Get the type of WiFi parameter
Return：	0	success
	<0	fail
Remark：		

CONFIDENTIAL

7.9.11 Lib_WifiGetInfo

Prototype	**int Lib_WifiGetInfo(NET_INFO eType, unsigned char *pucStr, unsigned short *pusLen);**	
Description	Get WiFi parameters	
Parameters	**eType[in]**	Parameter type ID
	pucStr[in]	Parameter buffer
	pusLen[in][out]	Input parameter：buffer length Output parameter：get parameter length
Return	0	success
	<0	failed
Remark	The parameter type ID, please refer to the NET_INFO enumeration type definition.	

7.9.12 Lib_WifiSetInfo

Prototype	**int Lib_WifiSetInfo(NET_INFO eType, unsigned char *pucStr, unsigned short usLen);**	
Description	set WiFi relevant parameters	
Parameters	**eType[in]**	Parameter type ID
	pucStr[in]	Parameter buffer
	usLen[in]	Parameter length
Return	0	success
	<0	failed
Remark	The parameter type ID, please refer to the NET_INFO enumeration type definition.	

7.9.13 Lib_WifiGetVersion

Prototype：	**int Lib_WifiGetVersion(char *ver, unsigned short *len);**	
Description	Get WiFi version	
Parameters	**ver[out]**	Get WiFi version information buffer
	len[out]	Get the length of WiFi version information
Return:	0	success
	<0	fail

CONFIDENTIAL

Remark:	

7.10 Battery Manage Interfaces

7.10.1 Lib_BatRefreshDisCapacity

Prototype	**void Lib_BatRefreshDisCapacity(void);**
Description	Calculates the battery capacity.
Parameters	No input parameters.
Return	N/A
Remark	1.This function shall be called continuously in the application, for example, in an idle task or in the main loop. The implementation ensures that it runs at the required interval, so the application does not need to control the calling period. 2.The application shall not remove or change the calling of this function. 3.The battery capacity calculation is implemented in the demo, which calls this function continuously. The application shall not modify it. Refer to the demo for the correct usage. 4.After this function is called, the calculated capacity can be obtained by Lib_BatGetCapacity.

7.10.2 Lib_BatGetCapacity

Prototype	**unsigned char Lib_BatGetCapacity(void);**
Description	Gets the current battery capacity.
Parameters	No input parameters.
Return	Current battery capacity, in percent.
Remark	The returned value ranges from 0 to 100, where 0 means empty and 100 means full.

7.10.3 Lib_BatDischargeLowCap

Prototype	**unsigned char Lib_BatDischargeLowCap(void);**	
Description	Determines whether a forced shutdown is required in the discharging state.	
Parameters	No input parameters.	
Return	0	No forced shutdown is required.

CONFIDENTIAL

	1	A forced shutdown is required.
Remark	When this function returns 1, the application shall save the data that needs to be retained and shut down the device by calling Lib_ShutDown.	

7.10.4 Lib_BatGetPowerOffVol

Prototype	**unsigned long Lib_BatGetPowerOffVol(void);**
Description	Gets the configured shutdown voltage.
Parameters	No input parameters.
Return	Configured shutdown voltage, in millivolts (mV).
Remark	The returned value is the configured shutdown voltage threshold. When the battery voltage is lower than this value, the device shall be shut down.

7.10.5 Lib_BatGetVoltage

Prototype	**unsigned long Lib_BatGetVoltage(void);**
Description	Gets the battery voltage.
Parameters	No input parameters.
Return	Current battery voltage, in millivolts (mV).
Remark	

7.10.6 Lib_BatGetChargeState

Prototype	**int Lib_BatGetChargeState(void);**	
Description	Gets the battery charging state.	
Parameters	No input parameters.	
Return	0	The battery is not being charged.
	1	The battery is being charged.
Remark	1.When this function returns 0, it means that the battery is not being charged. This may be because no external power adapter is connected or because the battery is fully charged. 2.The application can call Lib_ExtPower to check whether the external power adapter is connected, and Lib_BatGetCapacity to check whether the battery is fully charged.	

CONFIDENTIAL

7.10.7 Lib_BatGetLibVersion

Prototype：	**void Lib_BatGetLibVersion(char *paucLibVer, unsigned short *pusLen);**	
Description	Get WiFi version	
Parameters	**ver[out]**	Get version information buffer
	len[out]	Get the length of version information
Return:	N/A	
Remark:		

7.11 Network Communication Interfaces

7.11.1 Lib_NetSocket

Prototype	**int Lib_NetSocket(int domain,int type,int protocol);**	
Description	Create a socket	
Parameters	**domain[in]**	Protocol family. Multi-protocol-family feature is not implemented in current version. Must be set to 0, reserved for future extension.
	type[in]	Socket type. Multi-socket-type feature is not implemented in current version. Must be set to 0, reserved for future extension.
	Protocol[in]	Socket mode. Specifies to create plain-TCP socket or TLS encrypted socket. Refer to NETWORK-TYPE enumeration values.
return	>=0	Success, returns socket handle.
	<0	Creation failed.
Remark	eg: handle = Lib_NetSocket(0, 0, NETWORK_TCP);	

7.11.2 Lib_NetConnect

Prototype	**int Lib_NetConnect(int handle, const char *host, unsigned short port);**	
Description	Network TCP/SSL Connect	
Parameters	**handle[in]**	Socket Number

CONFIDENTIAL

	host[in]	The server address to be connected can only be an IP address, not the domain name string.
	port[in]	The port to be connected of the server
return	0	Connect success
	<0	Connect failed
Remark	eg: ret = Lib_NetConnect(handle, "165.22.21.77", 443);	

7.11.3 Lib_NetSend

Prototype	**int Lib_NetSend(int handle, const void *data, int dataLen);**	
Description	Network sending data	
Parameters	**handle[in]**	Socket Number
	data[in]	Data cache pointer to be sent
	dataLen[in]	The length of data to be sent
return	>=0	The length of data sent
	<0	Send Fail
Remark	Eg: ret = Lib_NetSend(handle, buf, bufLen);	

7.11.4 Lib_NetRecv

Prototype	**int Lib_NetRecv(int handle, void *buff, int buffSize);**	
Description	Receive data from the network	
Parameters	**handle[in]**	Socket Number
	buff[out]	Cache pointer to receive data
	buffSize[in]	Maximum space of receive data cache
return	>=0	The length of received data
	<0	Receive fail
Remark	eg: ret = Lib_NetRecv(handle, buf, bufLen);	

7.11.5 Lib_NetClose

Prototype	**void Lib_NetClose(int handle);**

CONFIDENTIAL

Description	Close network connection	
Parameters	**handle[in]**	Socket Number
return	N/A	
Remark	eg: Lib_NetClose(handle);	

7.11.6 Lib_NetGetStatus

Prototype	**int Lib_NetGetStatus(int handle);**	
Description	Get network connection status	
Parameters	**handle[in]**	Socket Number
return	>=0	Current status of the network connection, refer to socket_status_t enumeration values.
Remark	eg: ret = Lib_NetGetStatus(handle);	

7.11.7 Lib_NetSetCfg

Prototype	**int Lib_NetSetCfg(int handle, SOCK_CFG_TYPE_E type, unsigned long param);**	
Description	Set network connection parameters	
Parameters	**handle[in]**	Set network connection parameters
	type[in]	Parameters type, refer to SOCK_CFG_TYPE_E typedef
	param[in]	The value of the parameter to be set
return	=0	Set Success
	<0	Set Fail
Remark	eg: Lib_NetSetCfg(handle, TLSSOCK_CFG_TYPE_SNI, domain);	

7.11.8 Lib_NetDns

Prototype	**int Lib_NetDns(char *name, char *result, unsigned short *pusIpLen);**	
Description	Domain name resolution	
Parameters	**name[in]**	Domain name
	result[out]	IP data from domain name resolution

CONFIDENTIAL

	pusIpLen[in][out]	Input: Maximum length of cache of IP data Output: IP data length of resolution
return	=0	Parse Success
	<0	Parse Fail
Remark	eg: ret = Lib_NetDns(domain, ip, &ipLen);	

7.11.9 Lib_NetOpenEx

Prototype	**int Lib_NetOpenEx(int *handle, NETWORK_TYPE protocol, int timeout, int (*Call)(int timeout, int ct));**	
Description	Extended open network socket. Create network instance, support network-type selection, timeout configuration and event callback.	
Parameters	**handle[out]**	Output socket handle. Valid handle is returned when API succeeds.
	protocol[in]	Network type. Select plain-TCP or TLS encrypted channel, refer to NETWORK-TYPE enumeration values.
	timeout[in]	Timeout value in milliseconds.
	Call[in]	Event callback function pointer. Can be NULL if callback is not required.
return	>=0	success
	<0	fail
Remark	eg: #define NET_MAX_INIT_TIME (180*1000) int ret = Lib_NetOpenEx(handle, NETWORK_TCP, NET_MAX_INIT_TIME, NULL);	

7.11.10 Lib_NetConnectEx

Prototype	**int Lib_NetConnectEx(int handle, char *server, unsigned short port,int timeout,int (*Call)(int timeout, int ct));**	
Description	Extended socket connect. Establish connection to specified server address and port, support timeout configuration and event callback.	
Parameters	**handle[in]**	Valid network handle obtained from Lib_NetOpenEx.
	server[in]	Server IP address or domain-name string.
	port[in]	Server port number.
	timeout[in]	Connection timeout in milliseconds.

CONFIDENTIAL

	Call[in]	Event callback function pointer. Can be NULL if callback is not required.
return	0	success
	<0	fail

Remark	eg:
	#define NET_MAX_TCPLINK_TIME (15*1000)
	int Net_OpenCb(int timeout, int ct)
	{
	}
	ret = Lib_NetConnectEx(handle, "165.22.21.77", 443, NET_MAX_TCPLINK_TIME, Net_OpenCb);	

7.11.11 Lib_NetStatusEx

Prototype	**int Lib_NetStatusEx(int handle);**	
Description	Get network connection status	
Parameters	**handle[in]**	Valid network handle obtained from Lib_NetOpenEx.
return	>=0	Current status of the network connection, refer to socket_status_t enumeration values.
Remark	eg: ret = Lib_NetStatusEx(handle);	

7.11.12 Lib_NetReadEx

Prototype	**int Lib_NetReadEx(int handle, unsigned char *buff, int buff_size, int timeout, int (*Call)(int timeout, int ct));**	
Description	Extended network read. Receive data from socket, support timeout configuration and event callback.	
Parameters	**handle[in]**	Valid network handle obtained from Lib_NetOpenEx.
	buff[out]	Receive data buffer.
	buff_size[in]	Maximum byte length of receive buffer.
	timeout[in]	Receive timeout in milliseconds.
	Call[in]	Event callback function pointer, can be NULL. Callback.
return	>=0	Actual received byte count;
	<0	fail
Remark	eg: #define NET_MAX_RX_TIME (15*1000) int Net_ReadCb(int timeout, int ct) {	

CONFIDENTIAL

	} ret = Lib_NetReadEx(handle, buf, bufLen, NET_MAX_RX_TIME, Net_ReadCb);

7.11.13 Lib_NetWriteEx

Prototype	**int Lib_NetWriteEx(int handle, unsigned char *buff, int len, int (*Call)(void));**	
Description	Extended network write. Send data to socket, support callback without input parameters.	
Parameters	**handle[in]**	Valid network handle obtained from Lib_NetOpenEx.
	buff[in]	Data buffer to send.
	len[in]	Byte length of data to send.
	Call[in]	Event callback function pointer, can be NULL. Callback.
return	0	success
	<0	fail
Remark	Eg: ret = Lib_NetWriteEx(handle, buf, bufLen, NULL);	

7.11.14 Lib_NetCloseEx

Prototype	**int Lib_NetCloseEx(int handle);**	
Description	Extended socket close. Release network related resources.	
Parameters	**handle[in]**	Valid network handle obtained from Lib_NetOpenEx.
return	0	success
	<0	fail
Remark	eg: ret = Lib_NetCloseEx(handle);	

7.11.15 Lib_NetResetEx

Prototype	**int Lib_NetResetEx(int timeout);**	
Description	Reset entire network module. Disconnect all connections and reclaim resources, support reset timeout configuration.	
Parameters	**timeout[in]**	Reset operation timeout in milliseconds.
return	0	success

CONFIDENTIAL

	<0	fail
Remark	eg: Ret = Lib_NetResetEx(3000);	

7.12 Pinpad Interface

7.12.1 Lib_PciGetMac

Prototype	**int Lib_PciGetMac(unsigned char ucMacKeyId, unsigned short usInLen, unsigned char *pucInData, unsigned char *pucMacOut, unsigned char ucMode);**	
Function	apply the specified MAC working key to perform MAC operations on the data and return the encrypted result.	
Parameters	**ucMacKeyId [in]**	specify the MAC key SN, The value range is 2~7 or 12~15, other values are illegal.
	usInLen[in]	count the data length of MAC message (the maximum length cannot exceed 1024 bytes)
	***pucInData[in]**	count the data content of MAC messages
	***pucMacOut [out]**	return the MAC message encryption result
	ucMode[in]	0=Algorithm 1
		1=Algorithm 2
		2=Algorithm 3
		3=Algorithm 4
Return	(0)	success
	(-7001)	key type error
	(-7002)	key check value wrong
	(-7003)	illegal key index
	(-7004)	illegal key length
	(-7011)	key not exist
	other	error

CONFIDENTIAL

Remark	Divide the message data into 8-byte blocks. If the length of the last block is less than 8 bytes, pad 0x00 at the end. Algorithm 1: Encrypt BLOCK1 with the MAC key using DES/3DES. Perform bit-wise XOR between the encrypted result and BLOCK2, then encrypt the XOR result again with the MAC key using DES/3DES. Repeat this process iteratively to obtain an 8-byte encrypted result. Algorithm 2: Compute bit-wise XOR of BLOCK1 and BLOCK2, then XOR the output with BLOCK3, and continue in this manner. An 8-byte XOR result is obtained finally, which shall be encrypted with the MAC key using DES/3DES. Algorithm 3: Encrypt BLOCK1 with the MAC key using DES. Perform bit-wise XOR between the encrypted result and BLOCK2, followed by another DES encryption with the MAC key. Apply 3DES encryption for the final round to generate an 8-byte encrypted result. Algorithm 4: Perform bit-wise XOR between BLOCK1 and BLOCK2, then XOR the result with BLOCK3, and proceed sequentially to get an 8-byte XOR result. Convert this 8-byte XOR result into 16-character hexadecimal data. Take the first 8 bytes and encrypt them with the MAC key using DES/3DES. XOR the encrypted output with the subsequent 8 bytes. Apply double-length encryption to the XOR result using the MAC key. Convert the operation output into 16-character hexadecimal data, and take the first 8 bytes as the MAC value.

7.12.2 Lib_PciAlgDes

Prototype	**int Lib_PciAlgDes(unsigned char ucKeyId, unsigned char* pucIv, unsigned char *pucInData, unsigned short usInLen, unsigned char *pucOutData, unsigned char ucMode);**	
Function	Specify the key to perform DES operation, and select ECB or CBC modes.	
Parameters	**ucKeyId[in]**	key index, the value range is 0~16 (except reserved key ID), other values are illegal.
	pucIv[in]	Initial vector, CBC mode must be passed in
	pucInData[in]	Data buffer to be encrypted or decrypted
	usInLen[in]	Length of data to be encrypted or decrypted
	pucOutData[out]	The returned data result after encryption or decryption processing
	ucMode[in]	Bit2: 0x00: encryption mode 0x04: decryption mode Bit3: 0x00: ECB mode 0x08: CBC mode
Return	(0)	success
	<0	fail
Remark	Attention: Select the corresponding calculation mode according to the key length by the passed in key ID	

CONFIDENTIAL

7.12.3 Lib_PciAlgAes

Prototype	**int Lib_PciAlgAes(unsigned char ucKeyId, unsigned char* pucIv, unsigned char *pucInData, unsigned short usInLen, unsigned char *pucOutData, unsigned char ucMode);**	
Function	Specify the key to perform AES operation, and select ECB or CBC modes.	
Parameters	**ucKeyId[in]**	key index, the value range is 0~16 (except reserved key ID), other values are illegal.
	pucIv[in]	Initial vector, CBC mode must be passed in
	pucInData[in]	buffer for data to be encrypted or decrypted
	usInLen[in]	Length of data to be encrypted or decrypted
	pucOutData[out]	The returned data result after encryption or decryption processing
	ucMode[in]	Bit2: 0x00: encryption mode 0x04: decryption mode Bit3: 0x00: ECB mode 0x08: CBC mode
Return	(0)	Success
	<0	failed
Remark	Notice:Select the corresponding calculation mode according to the key length pointed to by the incoming key ID.	

7.12.4 Lib_PciWriteKey

Prototype	**int Lib_PciWriteKey(unsigned char ucKeyId, unsigned char ucKeyLen, unsigned char *pucKey, unsigned char ucMKeyId, unsigned char* pucCheckValue, unsigned char ucCVLen);**	
Function	Write encryption key according to the specified key sequence number. it is recommended to use this function.	
Parameters	**ucKeyId[in]**	key ID, stored by specified ID
	ucKeyLen[in]	key length
	pucKey[in]	key
	ucMKeyId[in]	primary key, to protect write key
	pucCheckValue[in]	data encrypted with all 0s
	ucCVLen [in]	The Length of CheckValue
Return	(0)	success
	(-7001)	key type error
	(-7002)	key check value error

CONFIDENTIAL

	(-7003)	illegal key index
	(-7004)	illegal key length
	(-7012)	write key fail
	others	Other errors.
Remark		

7.12.5 Lib_PciGetPin

Prototype	**int Lib_PciGetPin(unsigned char ucPinKeyId, unsigned char *pucCardNo, unsigned char ucMode, unsigned char *pucPinBlock);**	
Function	Calculate PIN ciphertext block (PinBlock) according to card number, working mode and specified PIN key ID.	
Parameters	**ucPinKeyId[in]**	PIN encryption key ID, stored by specified ID
	pucCardNo[in]	Card number data buffer
	ucMode[in]	PIN encryption mode
	pucPinBlock[in/out]	Output buffer for PIN ciphertext block result
Return	(0)	success
	(-7001)	key type error
	(-7002)	key check value error
	(-7003)	illegal key index
	(-7004)	illegal key length
	(-7016)	parameter error
	others	Other errors.
Remark		

7.13 Files Operation

7.13.1 Lib_FileOpen

Prototype	**int Lib_FileOpen(char *pscFileName, int iMode);**	
Function	open or create current file	
Parameters	**pscFileName [in]**	file name, ending with '\x00' (length does not exceed 230 bytes)
	iMode[in]	file opening method, (defined in <fcntl.h>) O_RDONLY means that the file is opened for reading only O_RDWR means to open the file for reading/writing O_CREATE means to create a new file

CONFIDENTIAL

Return	>=0	success，return file handle number
	<0	failed
Remark	If file is to open in O_CREATE | O_RDWR, if the file already exists, open it will be opened in O_RDWR, otherwise new file will be created.	

7.13.2 Lib_FileRead

Prototype	**int Lib_FileRead(int slFileHandle, unsigned char *pucData, unsigned long ulLen);**	
Function	read file content	
Parameters	**slFileHandle[in]**	file handle number，Obtained by the Lib_FileOpenReturn value
	pucData[out]	data buffer area ， allocated by the application program.
	ulLen[in]	number of data bytes to read.
Return	>=0	read success，return actual bytes number read
	<0	failed
Remark		

7.13.3 Lib_FileWrite

Prototype	**int Lib_FileWrite(int slFileHandle, unsigned char *pucData, unsigned long ulLen);**	
Function	write file	
Parameters	**slFileHandle[in]**	file handle number，obtained by the Lib_FileOpenReturn value
	pucData[in]	data buffer to be written.
	ulLen[in]	number of data bytes to be written.
Return	>=0	write success，return actual bytes number write
	<0	failed
Remark		

7.13.4 Lib_FileClose

Prototype	**int Lib_FileClose(int slFileHandle);**	
Function	close file	
Parameters	**slFileHandle[in]**	file handle number，obtained by the Lib_FileOpenReturn value
Return	0	success
	<0	failed
Remark		

CONFIDENTIAL

7.13.5 Lib_FileSeek

Prototype	**int Lib_FileSeek(int slFileHandle, int slOffset, LFS_SEEK_FLAG fromWhere);**	
Function	Move the file read-write pointer position.	
Parameters	**slFileHandle[in]**	file handle number，obtained by the Lib_FileOpenReturn value
	slOffset [in]	the number of bytes from the position specified by fromwhere to the position to be moved (signed value)
	fromWhere[in]	can take a value from FILE_SEEK_CUR、FILE_SEEK_SET & FILE_SEEK_END。 FILE_SEEK_CUR *start from the current file pointer*; FILE_SEEK_SET *start from the beginning of the file*; FILE_SEEK_END *start from the end of the file.*
Return	0	success
	<0	failed
Remark		

7.13.6 Lib_FileSize

Prototype	**int Lib_FileSize(char *pscFileName);**	
Function	get file size	
Parameters	**pscFileName [in]**	file name, up to 230 characters, ends with '\x00'
Return	>=0	file size (bytes)
	<0	failed
Remark		

7.13.7 Lib_FileRemove

Prototype	**int Lib_FileRemove(const char *pscFileName);**	
Function	delete file	
Parameters	**pscFileName [in]**	file name, up to 230 characters, ends with '\x00'
Return	0	success
	<0	failed
Remark		

CONFIDENTIAL

7.13.8 Lib_FileFreeSize

Prototype	long Lib_FileFreeSize(void);	
Function	read the remaining storage space of the file system	
Parameters	No input parameters.	
Return	return the total remaining space of the file system (number of bytes)	
Remark		

7.13.9 Lib_FileMkdir

Prototype	int Lib_FileMkdir(const char *pscName, unsigned char ucMode);	
Description	Create a directory	
Parameters	pscName[in]	Directory name， which ends with ‘\x00’ (The length cannot exceed 230 bytes)
	ucMode[in]	Create permissions of directory， represented by 0 generally
Return	0	success
	<0	failed
Remark		

7.13.10 Lib_FileOpendir

Prototype	int Lib_FileOpendir(const char *pscPath)	
Description	Open a directory	
Parameters	pscPath [in]	Directory name，which ends with ‘\x00’(The length cannot exceed 230 bytes)
Return	>=0	success，return directory handle number
	<0	failed
Remark		

7.13.11 Lib_FileReaddir

Prototype	int Lib_FileReaddir(int fd, char *namebuf)	
Description	Read file name in the folder	
Parameters	fd [in]	Directory handle
	namebuf [in]	File nams storage data cache
Return	0	success
	<0	failed
Remark		

CONFIDENTIAL

7.13.12 Lib_FileClosedir

Prototype	int Lib_FileClosedir(int fd)	
Description	Close a directory	
Parameters	fd [in]	Directory or Directory file handle
Return	0	success
	<0	failed
Remark		

7.13.13 Lib_FileGetAllFiles

Prototype	int Lib_FileGetAllFiles(const char *pscPath, char *buf, int size)	
Description	Read all files's name in the directory	
Parameters	pscPaths [in]	directory path name
	buf[outs]	Data cache for storing all file names
	size[in]	Maximum cache size
Return	0	success
	<0	failed
Remark		

7.13.14 Lib_FileClearPathAllFiles

Prototype	int Lib_FileClearPathAllFiles(const char *pscPath)	
Description	Clear all files in the directory	
Parameters	pscPath [in]	directory path name
sReturn	0	success
	<0	failed
Remark		

7.14 TMS Upgrade

7.14.1 Lib_TmsDeleteSoft

Prototype	void Lib_TmsDeleteSoft(DOWNLOAD_MODULE module);

CONFIDENTIAL

Function	TMS software deletion. Delete the software/firmware corresponding to the specified download module.	
Parameters	**module[in]**	Download module to be deleted, refer to DOWNLOAD_MODULE enumeration.
Return	0	success
	<0	fail

Remark	typedef enum
	{
	DL_GPRS = 0, //GPRS module
	DL_WIFI, //WIFI module
	DL_SCHIP, //SE
	DL_FILE, //File system
	DL_FIRMWARE, //firmware
	} DOWNLOAD_MODULE;	

7.14.2 Lib_TmsDownloadRemote

Prototype	**short Lib_TmsDownloadRemote(NET_MODE netMode,DOWNLOAD_MODULE module,int timeout,unsigned char *updateFlag);**	
Function	Application firmware verification.	
Parameters	**netMode[in]**	Network mode, refer to NET_MODE enumeration definition.
	module[in]	Download module to be updated, refer to DOWNLOAD_MOUDLE enumeration.
	timeout[in]	Remote download timeout in milliseconds.
	updateFlag[out]	Output update status flag.
Return	0	success
	<0	fail
Remark	Currently, it supports upgrading OS firmware, application firmware, WiFi firmware, SE security-chip firmware and file system. Refer to the demo file Tms_Moudle.c for detailed upgrade procedures.	

7.14.3 Lib_TmsGetLibVersion

Prototype	**void Lib_TmsGetLibVersion(char *paucLibVer, unsigned short *pusLen);**	
Function	Get TMS library version. Output version string and the length of version string.	
Parameters	**paucLibVer[out]**	Output buffer for library version string.
	pusLen[in/out]	Input: maximum buffer length; Output: actual byte length of version string.
Return	0	success
	<0	fail
Remark		

CONFIDENTIAL

7.15 EMV Interface

7.15.1 Lib_EmvKernelInit

Prototype	int Lib_EmvKernelInit(PST_KERNEL_CALLBACK pCallback);	
Function	EMV kernel initialization	

Parameters	pCallback [in]	EMV kernel callback function;
		The structure that needs to contain the callback function in pCallback is as follows:ST_KERNEL_CALLBACK stEMVCallback =
		{
		EmvGetTerminalCapk,
		EmvGetTerminalAID,
		EmvCertRevolutionCheck,
		EmvExceptionFileCheck,
		EmvDispMsg,
		EmvAppConfirm,
		EmvCardNoConfirm,
		EmvInputPIN,
		};

Return	0	success
	<0	fail
Remark		

7.15.2 Lib_EmvProcess

Prototype	int Lib_EmvProcess(const PST_KERNEL_TRANS pstKernelTrans);	
Function	EMV kernel process	
Parameters	pstKernelTrans [in]	transaction parameters
Return	0	success
	<0	fail
Remark		

7.15.3 Lib_EmvAfterOnlineProcess

Prototype	int Lib_EmvAfterOnlineProcess(unsigned char ucOnlineRes, unsigned char *pucData, unsigned int ulLen);
Function	Secondary card reading process after online

CONFIDENTIAL

Parameters	ucOnlineRes [in]	Online result: 0: Online failure 1: Online success
	pucOnlineRespData[in]	Online response data
	uiOnlineRespDataLen[in]	Online response data length
Return	0	success
	other	fail
Remark	Used to process response data after online	

7.15.4 Lib_EmvGetKernelProcessResult

Prototype	int Lib_EmvGetKernelProcessResult(PST_KERNEL_RESULT pstResult);	
Function	Get EMV kernel process results	
Parameters	pstResult[out]	EMV kernel process result
Return	0	success
	<0	fail
Remark		

7.15.5 Lib_EmvGetTlv

Prototype	int Lib_EmvGetTlv(unsigned int uiTag, unsigned char *pucValue, unsigned int uiSize, unsigned int *puiLenth);	
Function	Get TLV data from EMV kernel	
Parameters	uiTag[in]	Get the TAG of TLV format data
	pucValue[out]	Data buffer
	uiSize[in]	Data buffer size
	puiLenth[out]	Get the length of the data
Return	0	success
	<0	fail
Remark		

CONFIDENTIAL

7.15.6 Lib_EmvSetTlv

Prototype	int Lib_EmvSetTlv(unsigned int uiTag, unsigned char *pucValue, unsigned int uiLenth);	
Function	Set data to EMV kernel	
Parameters	uiTag[in]	Set the TAG of TLV format data
	pucValue[in]	Data to be set
	uiLenth[in]	Data length to be set
Return	0	success
	<0	fail
Remark		

7.15.7 Lib_EmvGetEcBalance

Prototype	int Lib_EmvGetEcBalance(unsigned char ucChannelType, SDK_ST_EMV_EC *pstEmvEc);	
Function	Get the balance of electronic cash	
Parameters	ucChannelType[in]	Card channel type: 0: contact card 1: contactless card
	pstEmvEc[out]	Output parameter, pointer to electronic-cash structure, returns balance and related data
Return	0	success
	<0	fail
Remark		

7.15.8 Lib_EmvGetLibVersion

Prototype	void Lib_EmvGetLibVersion(char *paucEmvLibVer, unsigned short *pusLen);	
Function	Get the version number of EMV library	
Parameters	paucEmvLibVer[out]	Output buffer for version string, memory allocated by caller
	pusLen[in/out]	IN: input buffer size; OUT: actual length of returned version string
Return	0	success
	<0	fail
Remark		

CONFIDENTIAL

7.15.9 Lib_EarseTerminalAIDandCapk

Prototype	int Lib_EarseTerminalAIDandCapk(unsigned char ucType)	
Function	erase all AIDs and CAPKs in the terminal	
Parameters	ucType[in]	The value is one of the macro TERMINAL_AID or the macro TERMINAL_CAPK
Return	0	success
	<0	fail
Remark	The macro is defined in LibSysapi.h. When the ucType is TERMINAL_AID, the AID information in the device is erased; when the ucType is TERMINAL_CAPK, all CAPK information in the device is erased	

7.15.10 Lib_GetAIDandCapkTotal

Prototype	unsigned char Lib_GetAIDandCapkTotal(unsigned char ucTypes)	
Function	get the number of AID and CAPK saved in the terminal	
Parameters	ucTypes[in]	the value is one of the macro TERMINAL_AID or the macro TERMINAL_CAPK
Return	>0	the number of obtained AID and CAPK
	=0	there is no AID and CAPK information in the device
Remark	Note: The macro is defined in LibSysapi.h. When the value of ucType is TERMINAL_AID, the information about the number of AIDs in the device is obtained; when the value of ucType is TERMINAL_CAPK, the information about the number of CAPKs stored in the device is got	

7.15.11 Lib_GetTerminalAID

Prototype	int Lib_GetTerminalAID(unsigned int uiAidIndex, unsigned char *pucTerAid, unsigned int uiSize, unsigned int *puiTerAidLen)	
Function	callback function to get terminal AID	
Parameters	uiAidIndex[in]	AID index, starting from 0
	pucTerAid[out]	Terminal AID, TLV format data
	uiSize[in]	Terminal AID buffer size
	puiTerAidLen[out]	Terminal AID data length
Return	0	success
	<0	fail

CONFIDENTIAL

Remark	

7.15.12 Lib_GetTerminalCapk

Prototype	short Lib_GetTerminalCapk(const unsigned char pucRid[5], unsigned char ucCapkIndex, unsigned char *pucCapk, unsigned short *pusCapkLen, unsigned char *pucExp, unsigned char *pucExpLen)	
Function	Get the callback function of the terminal CAPK	
Parameters	pucRid[in]	RID, obtained from data whose TAG is 4F
	ucCapkIndex[in]	public key index
	pucCapk[out]	public key
	pusCapkLen[in/out]	IN- Public key Buff length, OUT- Public key actual length
	pucExp[in]	public key exponent
	pucExpLen[in/out]	IN- Public key exponent Buff length, OUT- Public key exponent actual length
Return	0	success
	<0	fail
Remark		

7.15.13 Lib_SetTerminalAIDAndCapk

Prototype	short Lib_SetTerminalAIDAndCapk(unsigned char *pData, unsigned short DataLen, unsigned char ucType)	
Function	add AID and CAPK to the device	
Parameters	pData[in]	AID or CAPK to be added
	DataLen[in]	effective length of pData
	ucType[in]	the value is one of the macro TERMINAL_AID or the macro TERMINAL_CAPK
Return	0	success
	-1	fail
Remark		

CONFIDENTIAL

7.16 OS Interface (OSI) APIs

7.16.1 Lib_osiThreadCreate

Prototype	**void* Lib_osiThreadCreate(const char * name, osiThreadEntry_t entry, void * argument, unsigned long priority, unsigned long stack_size, unsigned long event_count);**	
Function	Creates a new thread and returns a thread handle. The returned handle is used for subsequent suspend, resume, and delete operations on the thread.	
Parameters	**name [in]**	Thread name, used for debugging and identification. Does not affect thread functionality.
	entry[in]	Pointer to the thread entry function. Execution starts from this function after the thread is created.
	argument[in]	Argument passed to the thread entry function. Pass NULL when no argument is required.
	priority [in]	Thread priority. A higher value indicates a higher priority.
	stack_size[in]	Stack size of the thread, in bytes.
	event_count[in]	Number of events available to the thread.
Return	Non-NULL	Thread created successfully; the thread handle is returned.
	NULL	Thread creation failed.
Remark	1.stack_size shall be set to a sufficient value; insufficient stack space will cause abnormal thread behavior. 2.When the thread is no longer required, Lib_osiThreadDelete shall be called to release the resources it occupies. 3.Do not delete a NULL handle or an already deleted handle. Do not delete the same handle more than once.	

7.16.2 Lib_osiThreadDelete

Prototype	**void Lib_osiThreadDelete(void * handle);**	
Function	Deletes the specified thread and releases the system resources it occupies.	
Parameters	**handle[in]**	**Thread handle returned by Lib_osiThreadCreate.**
Return	N/A	
Remark	1.The handle becomes invalid immediately after deletion and shall not be used for any further operation. 2.Ensure that the thread has completed execution or is no longer required before deleting it.	

CONFIDENTIAL

7.16.3 Lib_osiThreadCurrent

Prototype	void * Lib_osiThreadCurrent(void);	
Function	Returns the handle of the thread that is currently calling this function.	
Parameters	No input parameters.	
Return	Not NULL	Handle of the current thread.
	NULL	Failed to obtain the handle, or the caller is not in a thread context (for example, called from an ISR).
Remark		

7.16.4 Lib_osiMutexCreate

Prototype	void *Lib_osiMutexCreate(void);	
Function	Creates a mutex and returns its handle. A mutex is used to protect shared resources from being accessed by multiple threads simultaneously.	
Parameters	No input parameters.	
Return	Not NULL	Mutex created successfully; the mutex handle is returned.
	NULL	Mutex creation failed.
Remark	When the mutex is no longer required, Lib_osiMutexDelete shall be called to release the resources it occupies.	

7.16.5 Lib_osiMutexDelete

Prototype	void Lib_osiMutexDelete(void *mutex);	
Function	Deletes the specified mutex and releases the system resources it occupies.	
Parameters	mutex[in]	Mutex handle returned by Lib_osiMutexCreate.
Return	N/A	
Remark	1.The handle becomes invalid immediately after deletion and shall not be used for any further operation. 2.Do not delete a mutex that is currently locked by any thread. Ensure that no thread holds or waits for the mutex before deleting it. 3.Do not delete a NULL handle or an already deleted handle, and do not delete the same handle more than once.	

7.16.6 Lib_osiMutexLock

Prototype	void Lib_osiMutexLock(void *mutex);	
Function	Acquires the specified mutex. If the mutex is already held by another thread, the calling thread blocks until the mutex becomes available.	
Parameters	mutex[in]	Mutex handle returned by Lib_osiMutexCreate.

CONFIDENTIAL

Return	N/A
Remark	1.This function blocks indefinitely when the mutex is unavailable. Use Lib_osiMutexTryLock when a bounded wait is required. 2.Every successful Lib_osiMutexLock shall be paired with a Lib_osiMutexUnlock call.

7.16.7 Lib_osiMutexTryLock

Prototype	**bool Lib_osiMutexTryLock(void *mutex, unsigned long timeout);**	
Function	Attempts to acquire the specified mutex. If the mutex is not immediately available, the calling thread waits until the mutex becomes available or the specified timeout expires.	
Parameters	**mutex[in]**	Mutex handle returned by Lib_osiMutexCreate.
	timeout[in]	Maximum time to wait for the mutex, in ticks. See the note below for the tick duration.
Return	TRUE	The mutex was acquired successfully.
	FASLE	The mutex could not be acquired within the specified timeout.
Remark	Every call that returns true shall be paired with a Lib_osiMutexUnlock call. When the return value is false, the mutex was not acquired and shall not be unlocked.	

7.16.8 Lib_osiMutexUnlock

Prototype	**void Lib_osiMutexUnlock(void *mutex);**	
Function	Releases the specified mutex, allowing another waiting thread to acquire it.	
Parameters	**mutex[in]**	Mutex handle returned by Lib_osiMutexCreate.
Return	N/A	
sRemark	1.This function shall only be called by the thread that currently holds the mutex. 2.Do not unlock a mutex that is not locked, and do not unlock the same mutex more than once without acquiring it again. 3.Keep the code between lock and unlock as short as possible to avoid degrading system concurrency.	

7.16.9 Lib_osiSemaphoreCreate

Prototype	**void* Lib_osiSemaphoreCreate(unsigned long max_count, unsigned long init_count);**	
Function	Creates a counting semaphore with the specified maximum count and initial count, and returns its handle.	
Parameters	**max_count[in]**	Maximum count value of the semaphore. This is the upper limit the semaphore can reach.

CONFIDENTIAL

	init_count[in]	Initial count value of the semaphore.
Return	Not NULL	Semaphore created successfully; the semaphore handle is returned.
	NULL	Semaphore creation failed.
Remark	1.Semaphore creation failed. 2.When the semaphore is no longer required, Lib_osiSemaphoreDelete shall be called to release the resources it occupies.	

7.16.10 Lib_osiSemaphoreDelete

Prototype	void Lib_osiSemaphoreDelete(void *sem);	
Function	Deletes the specified semaphore and releases the system resources it occupies.	
Parameters	sem[in]	Semaphore handle returned by Lib_osiSemaphoreCreate.
Return	N/A	
Remark	1.The handle becomes invalid immediately after deletion and shall not be used for any further operation. 2.Do not delete a semaphore while any thread is blocked waiting for it. 3.Do not delete a NULL handle or an already deleted handle, and do not delete the same handle more than once.	

7.16.11 Lib_osiSemaphoreRelease

Prototype	void Lib_osiSemaphoreRelease(void *sem);	
Function	Increments the count value of the specified semaphore. If any thread is waiting for the semaphore, one waiting thread is woken up.	
Parameters	sem[in]	Semaphore handle returned by Lib_osiSemaphoreCreate.
Return	N/A	
Remark		

7.16.12 Lib_osiSemaphoreAcquire

Prototype	void Lib_osiSemaphoreAcquire(void *semaphore);	
Function	Decrements the count value of the specified semaphore. If the count value is zero, the calling thread blocks until the semaphore is released by another thread or an ISR.	
Parameters	semaphore[in]	Semaphore handle returned by Lib_osiSemaphoreCreate.
Return	N/A	
Remark	1.This function blocks indefinitely when the semaphore is unavailable. Use Lib_osiSemaphoreTryAcquire when a	

CONFIDENTIAL

	bounded wait is required

7.16.13 Lib_osiSemaphoreTryAcquire

Prototype	**bool Lib_osiSemaphoreTryAcquire(void *semaphore, unsigned long timeout);**	
Function	Attempts to decrement the count value of the specified semaphore. If the count value is zero, the calling thread waits until the semaphore becomes available or the specified timeout expires.	
Parameters	**semaphore[in]**	Semaphore handle returned by Lib_osiSemaphoreCreate.
	timeout[in]	Maximum time to wait for the semaphore, in ticks. See the note below for the tick duration.
Return	TRUE	The semaphore was acquired successfully.
	FASLE	The semaphore could not be acquired within the specified timeout.
Remark		

7.16.14 Lib_osiCreatQueue

Prototype	**void * Lib_osiCreatQueue(unsigned long conut, unsigned long size);**	
Function	Creates a queue with the specified number of items and item size, and returns its handle. The queue is used to transfer data between threads or between an ISR and a thread in FIFO order.	
Parameters	**count[in]**	Number of items the queue can hold.
	size[in]	Size of each queue item, in bytes.
Return	Not NULL	Queue created successfully; the queue handle is returned.
	NULL	Queue creation failed.
Remark	1.All items in a queue shall have the same size, which is specified by size at creation time. 2.The total memory consumed is approximately cont * size bytes. Ensure that sufficient memory is available. 3.When the queue is no longer required, the corresponding delete function shall be called to release resources.	

7.16.15 Lib_osiQueueSend

Prototype	**int Lib_osiQueueSend(void *handle, void *item, unsigned long tick_to_wait);**
Function	Sends an item to the tail of the specified queue. If the queue is full, the calling thread waits until space becomes available or the specified timeout expires.

CONFIDENTIAL

Parameters	handle[in]	Queue handle returned by Lib_osiCreatQueue.
	item[in]	Pointer to the item to be sent. The data pointed to is copied into the queue, and its size shall be equal to the size specified at queue creation.
	tick_to_wait[in]	Maximum time to wait when the queue is full.
Return	>0	The item was sent successfully.
	others	The item could not be sent, for example because the queue was full and the timeout expired.
Remark	The item data is copied into the queue, so the buffer pointed to by item may be reused or released after this function returns.	

7.16.16 Lib_osiQueueReceive

Prototype	int Lib_osiQueueReceive(void *handle, void *item, unsigned long tick_to_wait);	
Function	Receives an item from the head of the specified queue. If the queue is empty, the calling thread waits until an item becomes available or the specified timeout expires.	
Parameters	handle[in]	Queue handle returned by Lib_osiCreatQueue.
	item [out]	Pointer to the buffer that receives the item. The buffer shall be large enough to hold one item, as specified by size at queue creation.
	tick_to_wait[in]	Maximum time to wait when the queue is empty.
Return	>0	An item was received successfully.
	others	No item was received, for example because the queue was empty and the timeout expired.
Remark	The buffer pointed to by item shall be allocated by the caller and large enough to hold one complete item.	

7.16.17 Lib_osiTimerCreate

Prototype	void *Lib_osiTimerCreate(unsigned long ms, bool periodic, osiCallback_t cb, void *ctx);	
Function	Creates a software timer with the specified period, operating mode, and callback function, and returns its handle. The timer shall be started by Lib_osiTimerStart before it begins to run.	
Parameters	ms[in]	Timer period, in milliseconds.
	periodic[in]	Operating mode of the timer. true — the timer restarts automatically after each expiry (periodic); false — the timer expires only once (one-shot).
	cb [in]	Callback function invoked when the timer expires.

CONFIDENTIAL

	ctx[in]	Context pointer passed to the callback function. Pass NULL when no context is required.
Return	Non-NULL	Timer created successfully; the timer handle is returned.
	NULL	Timer creation failed.
Remark	1.Creating a timer does not start it. Lib_osiTimerStart shall be called to start the timer. 2.The callback function is executed in the timer service context. Keep it as short as possible, and do not call blocking functions within the callback. 3.When the timer is no longer required, Lib_osiTimerDelete shall be called to release the resources it occupies.	

7.16.18 Lib_osiTimerDelete

Prototype	int Lib_osiTimerDelete(void *timer);	
Function	Deletes the specified timer and releases the system resources it occupies. The timer is stopped first if it is currently running.	
Parameters	timer[in]	Timer handle returned by Lib_osiTimerCreate.
Return	0	The timer was deleted successfully.
	<0	Failed to delete the timer.
Remark	1.The handle becomes invalid immediately after deletion and shall not be used for any further operation. 2.Do not delete a NULL handle or an already deleted handle, and do not delete the same handle more than once.	

7.16.19 Lib_osiTimerStart

Prototype	int Lib_osiTimerStart(void *timer);	
Function	Starts the specified timer. The timer begins counting from the period specified at creation, and invokes the callback function upon expiry.	
Parameters	timer[in]	Timer handle returned by Lib_osiTimerCreate.
Return	0	The timer was started successfully.
	<0	Failed to start the timer.
Remark		

7.16.20 Lib_osiTimerStop

Prototype	int Lib_osiTimerStop(void *timer);	
Function	Stops the specified timer. A stopped timer stops counting and does not invoke the callback function until it is started again.	
Parameters	timer[in]	Timer handle returned by Lib_osiTimerCreate.
Return	0	The timer was stopped successfully.
	<0	Failed to stop the timer.
Remark		

CONFIDENTIAL

7.17 Data Conversion

7.17.1 Lib_Asc2Bcd

Prototype	int Lib_Asc2Bcd(char *pcAsc, unsigned char *pucBcd, unsigned short usLen);	
Function	Converts an ASCII string to BCD data.	
Parameters	pcAsc[in]	Pointer to the source ASCII string.
	pucBcd[out]	Pointer to the buffer that receives the converted BCD data. The buffer shall be allocated by the caller.
	usLen[in]	Length of the source ASCII string.
Return	0	Success
	<0	Fail
Remark	1.The source string shall contain only numeric characters, that is, '0' to '9'. If it contains any non-numeric character, the conversion fails and a value less than 0 is returned. 2.The BCD buffer shall be allocated by the caller and shall be large enough to hold the converted data.	

7.17.2 Lib_Bcd2Asc

Prototype	int Lib_Bcd2Asc(unsigned char *pucBcd, char *pcAsc, unsigned short usLen);	
Function	Converts BCD data to an ASCII string. Each BCD byte is converted to two ASCII characters.	
Parameters	pcAsc[in]	Pointer to the source BCD data.
	pucBcd[out]	Pointer to the buffer that receives the converted ASCII string. The buffer shall be allocated by the caller.
	usLen[in]	Length of the source BCD data.
Return	0	Success
	<0	Fail
Remark	1.The ASCII buffer shall be allocated by the caller and shall be large enough to hold the converted string. 2.The converted string is not null-terminated. The application shall ensure the string terminator itself.	

7.17.3 Lib_Bcd2Int

Prototype	int Lib_Bcd2Int(unsigned char *pucBcd, unsigned char ucBcdLen);	
Function	Converts BCD data to an integer.	
Parameters	pucBcd[in]	Pointer to the source BCD data.

CONFIDENTIAL

	ucBcdLen[in]	Length of the source BCD data, in bytes.
Return	The converted integer value.	
Remark	The value represented by the BCD data shall not exceed the range of the int type; otherwise, the result is undefined.	

7.17.4 Lib_Int2Bcd

Prototype	int Lib_Int2Bcd(unsigned int uiNum, unsigned char *pucBcd, unsigned char ucBcdLen);	
Function	Converts BCD data to an ASCII string. Each BCD byte is converted to two ASCII characters.	
Parameters	uiNum[in]	Integer value to be converted.
	pucBcd[out]	Pointer to the buffer that receives the converted BCD data. The buffer shall be allocated by the caller.
	ucBcdLen[in]	Length of the BCD data to be generated, in bytes.
Return	0	Success
	<0	Fail
Remark	1.When uiNum is fully converted before ucBcdLen bytes are written, the remaining bytes are not modified. The application shall ensure the default value of the pucBcd buffer, for example, by initializing it to 0. 2.When uiNum requires more bytes than ucBcdLen, only the lowest ucBcdLen bytes are converted and the higher digits are discarded.	

7.17.5 Lib_HextoStr

Prototype	void Lib_HextoStr(unsigned char *src, unsigned char *dst, unsigned char Src_Len);	
Function	Converts hexadecimal data to its string representation, in which each byte is represented by two uppercase hexadecimal characters.	
Parameters	src[in]	Pointer to the source hexadecimal data.
	dst[out]	Pointer to the buffer that receives the converted string. The buffer shall be allocated by the caller.
	Src_Len[in]	Length of the source hexadecimal data, in bytes.
Return	0	Success
	<0	Fail
Remark	1.The converted characters are uppercase, that is, A to F are used for the values 10 to 15. 2.Each source byte is converted to two characters, so the dst buffer shall be at least 2 × Src_Len bytes. 3.The converted string is not null-terminated. The application shall ensure the string terminator itself.	

CONFIDENTIAL

7.17.6 Lib_AscNum2bcd

Prototype	**void Lib_AscNum2bcd(unsigned char *pucAsc, unsigned char *pucBcd, unsigned char ucAscLen, unsigned char *pucBcdLen, unsigned char ucAdd);**	
Function	Converts an ASCII numeric string to BCD data. If the length of the source string is odd, the low nibble of the last byte is filled with the value specified by ucAdd.	
Parameters	**pucAsc[in]**	Pointer to the source ASCII numeric string.
	pucBcd[out]	Pointer to the buffer that receives the converted BCD data. The buffer shall be allocated by the caller.
	ucAscLen[in]	Length of the source ASCII string.
	pucBcdLen[out]	Pointer to a variable that receives the length of the converted BCD data.
	ucAdd[in]	Fill value used to fill the low nibble of the last byte when the source length is odd.
Return	0	Success
	<0	Fail
Remark	1.The source string shall contain only numeric characters, that is, '0' to '9'. 2.When ucHexLen is odd, the low nibble of the last BCD byte is filled with ucAdd, and the high nibble contains the last numeric character.	

7.18 LVGL Custom Interfaces

7.18.1 Lib_LvglInit

Prototype	**void Lib_LvglInit(void);**
Function	LVGL Library Initialization
Parameters	No input parameters.
Return	N/A
Remark	

7.18.2 Lib_LvglGetFont

Prototype	**lv_font_t *Lib_LvglGetFont(unsigned short weight);**	
Function	Get LVGL Font through TTF	
Parameters	**weight[in]**	Font size

CONFIDENTIAL

Return	LVGL Font structure
Remark	

7.18.3 Lib_LvglTask

Prototype	**void Lib_LvglTask(void);**
Function	LVGL kernel process
Parameters	No input parameters.
Return	N/A
Remark	

7.18.4 Lib_LvglLock

Prototype	**void Lib_LvglLock(bool enable);**	
Function	Enables or disables LVGL mutex lock. Call this API before accessing LVGL API in multi-thread environment to guarantee thread-safe operation of LVGL graphics library.	
Parameters	**enable[in]**	true: acquire LVGL lock; false: release LVGL lock.
Return	N/A	
Remark	Must strictly follow acquire-release pairing rule. Do not repeatedly release lock without acquiring it.	

7.18.5 Lib_LvglSleep

Prototype	**void Lib_LvglSleep(void);**
Function	LVGL Library Initialization
Parameters	No input parameters.
Return	N/A
Remark	

7.18.6 Lib_LvglWakeUp

Prototype	**void Lib_LvglWakeUp(void);**
Function	LVGL Library Initialization
Parameters	No input parameters.
Return	N/A
Remark	

CONFIDENTIAL

7.19 LVGL Standard Interface

The library supports all standard interfaces of LVGL V8.3 version. Which can refer to https://docs.lvgl.io/8.3/ for usage of all standard interfaces.

7.20 cJSON Standard Interface

This SDK provides wrapper interfaces for the standard cJSON interfaces. The underlying library is cJSON 1.7.15. Each wrapper interface is a thin wrapper that directly invokes the corresponding standard cJSON interface; its parameters, return values, and semantics are identical to those of the standard interface.

For the detailed usage of each interface, including parameters, return values, and code examples, refer to the official cJSON documentation at https://github.com/DaveGamble/cJSON.

7.21 mxml Standard Interface

This SDK provides wrapper interfaces for the standard Mini-XML (mxml) interfaces. The underlying library is Mini-XML 4.0.3. Each wrapper interface is a thin wrapper that directly invokes the corresponding standard Mini-XML interface; its parameters, return values, and semantics are identical to those of the standard interface.

For the detailed usage of each interface, including parameters, return values, and code examples, refer to the official Mini-XML documentation at https://www.msweet.org/mxml.

7.22 Print Interface

7.22.1 Lib_PrnParaDefaultInit

Prototype	**int Lib_PrnParaDefaultInit(void);**	
Description	Reset printing parameters to their default values.	
Parameters	No input parameters.	
Return	>=0	Success
	<0	Fail

CONFIDENTIAL

Remark	Parameter initialization only; it restores font, zoom, line space, density and feed distance. It does NOT initialize the hardware — use Lib_PrnOpen() for that.

7.22.2 Lib_PrnOpen

Prototype	**int Lib_PrnOpen (void);**	
Description	Opens the printer module.	
Parameters	No input parameters.	
Return	>=0	Success
	<0	Fail
Remark	1.Must succeed before any print / feed / status call. 2.Call it in pair with Lib_PrnClose(); avoid opening repeatedly.	

7.22.3 Lib_PrnClose

Prototype	**int Lib_PrnClose(void);**	
Description	Closes the printer module.	
Parameters	No input parameters.	
Return	>=0	Success
	<0	Fail
Remark		

7.22.4 Lib_PrnCheckStatus

Prototype	**int Lib_PrnCheckStatus(void);**	
Description	Get the current printer status.	
Parameters	No input parameters.	
Return	>=0	Success
	-10	Paper shortage
	Other < 0	Error, currently undefined
Remark		

CONFIDENTIAL

7.22.5 Lib_PrnSetSpace

Prototype	**void Lib_PrnSetSpace(unsigned char ucSpace);**	
Description	Sets the line space of the printer module.	
Parameters	**ucSpace[in]**	Line space,unit is dot, range 1~16.
Return	N/A	
Remark		

7.22.6 Lib_PrnSetFont

Prototype	**int Lib_PrnSetFont(unsigned char ucFont, unsigned char ucZoomH, unsigned char ucZoomV);**	
Description	Sets the font size, horizontal zoom factor, and vertical zoom factor of the printer module.	
Parameters	**ucFont[in]**	Font size, range 8~64.
	ucZoomH[in]	Horizontal zoom factor, range 1~16.
	ucZoomV[in]	Vertical zoom factor, range 1~16.
Return	0	success
	<0	fail
Remark		

7.22.7 Lib_PrnGetFont

Prototype	**int Lib_PrnGetFont(unsigned char *pucFont , unsigned char *pucZoomH, unsigned char *pucZoomV);**	
Description	Gets the current font size, horizontal zoom factor, and vertical zoom factor of the printer module.	
Parameters	**ucFont[out]**	Output font size.
	ucZoomH[out]	Output horizontal zoom factor.
	ucZoomV[out]	Output vertical zoom factor.
Return	0	success
	<0	fail
Remark		

CONFIDENTIAL

7.22.8 Lib_PrnSetGrade

Prototype	**void Lib_PrnSetGrade(unsigned char ucGrade);**	
Description	Sets the density grade of the printer module.	
Parameters	**ucGrade[in]**	Input density grade, range 1~5, 1 is lightest, 5 is darkest.
Return	N/A	
Remark		

7.22.9 Lib_PrnGetGrade

Prototype	**unsigned char Lib_PrnGetGrade(void);**
Description	Gets the current density grade of the printer module.
Parameters	No input parameters.
Return	1~5: Current density grade, 1 is lightest, 5 is darkest.
Remark	

7.22.10 Lib_PrnSetForwardDistance

Prototype	**void Lib_PrnSetForwardDistance(unsigned short usPaperLen);**	
Description	Sets the default paper feed length when the printer module is closed.	
Parameters	**usPaperLen[in]**	Input paper feed length, usually in millimeters.
Return	N/A	
Remark	1.After setting, the printer module feeds paper by this length when it is closed. 2.This function sets the default paper feed length and does not feed paper immediately. To feed paper immediately, call Lib_PrnPaperForward. 3.The setting only affects the paper feed behavior when the printer module is closed later. It does not change already printed content.	

7.22.11 Lib_PrnPaperForward

CONFIDENTIAL

Prototype	**void Lib_PrnPaperForward(int iPaperDistance);**	
Description	Controls the printer module to feed paper by a specified distance.	
Parameters	**iPaperDistance[in]**	Input paper feed distance, unit is micrometer.
Return	N/A	
Remark		

7.22.12 Lib_PrnGetStrWidth

Prototype	**int Lib_PrnGetStrWidth(char *prnBuf, unsigned short prnLen, unsigned char ucFont, unsigned char ucZoomH);**	
Description	Calculates the print width of the specified string with the given font and horizontal zoom factor.	
Parameters	**prnBuf[in]**	Input string buffer.
	prnLen[in]	Input string length.
	ucFont[in]	Input font size, range 8~64.
	ucZoomH[in]	Input horizontal zoom, range 1~16 .
Return	>=0	Print width of the string, in dots.
	<0	fail
Remark		

7.22.13 Lib_PrnStr

Prototype	**int Lib_PrnStr(unsigned char *fmt, ...);**	
Description	Formats the output according to fmt and sends it to the printer module for printing. Supports variable arguments, similar to standard C printf.	
Parameters	**fmt[in]**	Format string, terminated by '\0'.
	...	Variable arguments, whose number and types are determined by the format specifiers in fmt.
Return	0	success
	<0	fail
Remark	1.Supports variable arguments. Refer to the printf() function in standard C. 2.Supports the '\n' (new line) and '\f' (form feed) control characters in the print string.	

CONFIDENTIAL

	3. If the print string exceeds the printable area, it will automatically wrap and continue printing. 4.The maximum processing buffer size of this function is 1500 bytes per call. 5.Do not use floating-point parameters with the '%f' format specifier. 6.Call this function after the printer module is successfully opened by Lib_PrnOpen.

7.22.14 Lib_PrnStrEx

Prototype	**int Lib_PrnStrEx(unsigned char mode, char *prnBuf, unsigned short prnLen);**	
Description	Prints a string in the specified mode, supporting alignment and inverted-color printing.	
Parameters	**mode[in]**	mode[in]: Print mode. Lower 4 bits: 0x00 left-aligned 0x01 center 0x02 right-aligned 0x03 justified (both ends) Upper 4 bits: 0x10 inverted-color printing (white text on black background)
	prnBuf[in]	Pointer to the string data to be printed.
	prnLen[in]	Length of the string data to be printed.
Return	0	success
	<0	fail
Remark	1.The lower 4 bits and upper 4 bits of mode can be combined. For example, 0x11 means center-aligned with inverted-color printing. 2.In justified mode (lower 4 bits of mode = 0x03), the string must be separated by "\r\n". The text before the delimiter is printed on the left, and the text after the delimiter is printed on the right. 3.In justified mode, if the total dot-matrix width of the left and right strings exceeds one line (384 dots), printing will fail. 4.The maximum processing buffer size of this function is 1500 bytes per call. 5.Call this function after the printer module is successfully opened by Lib_PrnOpen.	

7.22.15 Lib_PrnEsig

Prototype	**int Lib_PrnEsig(unsigned char *pucEsigData);**	
Description	Prints the electronic signature.	
Parameters	**pucEsigData[**	Prints the electronic signature.

CONFIDENTIAL

	in]	
Return	0	success
	<0	fail
Remark	1.The print data is returned by the Lib_TpGetESignData interface. 2.Call this function after the printer module is successfully opened by Lib_PrnOpen.	

7.22.16 Lib_PrnBmp

Prototype	**int Lib_PrnBmp(unsigned short usStartX, unsigned char *pucBmpData);**	
Description	Prints the electronic signature.	
Parameters	**usStartX[in]**	Starting X coordinate, usually in dots.range 0~383.
	pucBmpData[in]	Pointer to the BMP image data.
Return	0	success
	<0	fail
Remark	1.Prints a 1-bit monochrome, uncompressed BMP image. 2.Only 1-bit BMP with a black-and-white palette and BI_RGB (no compression) is supported. 3.Call this function after the printer module is successfully opened by Lib_PrnOpen.	

7.22.17 Lib_PrnXCode

Prototype	**int Lib_PrnXCode(unsigned short usStartX, char *prnBuf, unsigned short usXSize, unsigned short usYSize, unsigned char *pucLogo);**	
Description	Prints a QR code or barcode.	
Parameters	**usStartX[in]**	Starting X coordinate, in dots.range 0~383.
	prnBuf[in]	String data of the QR code or barcode to be printed.
	usXSize[in]	Width of the QR code or barcode, must be a multiple of 8.
	usYSize[in]	0 means print a QR code; non-zero means the height of the barcode, and must be a multiple of 8.
	pucLogo[in]	Pointer to the BMP image data of the logo in the center of the QR code. Can be NULL.

CONFIDENTIAL

Return	0	success
	<0	fail
Remark	1.The values of usXSize and usYSize must be multiples of 8. 2.When printing a barcode, usYSize must be non-zero. 3.When printing a QR code, usYSize must be 0. 4.pucLogo is only valid when printing a QR code and is used to overlay a logo image in the center of the QR code. It can be NULL when printing a barcode. 5.Call this function after the printer module is successfully opened by Lib_PrnOpen.	

CONFIDENTIAL

8 Appendix

CONFIDENTIAL